



**EDUCACIÓN**  
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO  
NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO DE LA PAZ  
DIVISIÓN DE ESTUDIOS DE POSGRADO E INVESTIGACIÓN  
MAESTRÍA EN SISTEMAS COMPUTACIONALES

**LOGÍSTICA DE ÚLTIMA MILLA EN REDES VIALES  
REALES: UN ENFOQUE REPRODUCIBLE PARA EL MDRP  
EN LA PAZ, B.C.S.**

QUE PARA OBTENER EL GRADO DE  
MAESTRO EN SISTEMAS COMPUTACIONALES

PRESENTA:

ALAN JOSÉ MARCOS SÁNCHEZ MARTÍNEZ

DIRECTORES DE TESIS:

DR. MARCO ANTONIO CASTRO LIERA

LA PAZ, BAJA CALIFORNIA SUR, MÉXICO, DICIEMBRE 2023.



# Índice general

|                                                                                       |          |
|---------------------------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                                                | <b>1</b> |
| 1. 1. Antecedentes . . . . .                                                          | 1        |
| 1. 1.1. De TSP a VRP . . . . .                                                        | 1        |
| 1. 1.2. VRP clásico y extensiones con restricciones . . . . .                         | 2        |
| 1. 1.3. Última milla y contexto urbano . . . . .                                      | 3        |
| 1. 1.4. Enrutamiento dinámico: DVRP y DARP . . . . .                                  | 4        |
| 1. 1.5. DVRP: Definición y características fundamentales . . . . .                    | 4        |
| 1. 1.6. DARP: Del transporte de pasajeros a la entrega de bienes . . . . .            | 5        |
| 1. 1.7. El valor de la información futura y el enfoque de Horizonte Rodante . . . . . | 6        |
| 1. 1.8. MDRP y políticas de despacho . . . . .                                        | 7        |
| 1. 1.9. Características compartidas con DVRP y DARP . . . . .                         | 7        |
| 1. 1.10. Definición y características del MDRP . . . . .                              | 7        |
| 1. 1.11. Enrutamiento realista en redes viales . . . . .                              | 8        |
| 1. 2. Planteamiento del problema . . . . .                                            | 9        |
| 1. 2.1. La ineficacia de las aproximaciones tradicionales . . . . .                   | 9        |
| 1. 2.2. La hostilidad topológica de La Paz, B.C.S. . . . .                            | 9        |
| 1. 2.3. La brecha en la eficiencia algorítmica . . . . .                              | 10       |
| 1. 3. Objetivos . . . . .                                                             | 10       |
| 1. 3.1. Objetivo general . . . . .                                                    | 10       |
| 1. 3.2. Objetivos específicos . . . . .                                               | 10       |
| 1. 4. Hipótesis . . . . .                                                             | 11       |
| 1. 5. Justificación . . . . .                                                         | 11       |
| 1. 5.1. Relevancia Científica . . . . .                                               | 11       |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| 1. 5.2. Aporte Tecnológico y Ciencia Abierta . . . . .                  | 12        |
| 1. 5.3. Impacto Económico y Social . . . . .                            | 12        |
| 1. 5.4. Delimitación frente a métodos de Aprendizaje Profundo . . . . . | 12        |
| 1. 6. Alcance y limitaciones . . . . .                                  | 13        |
| 1. 6.1. Alcance Geográfico y de Datos . . . . .                         | 13        |
| 1. 6.2. Alcance Funcional y Algorítmico . . . . .                       | 13        |
| 1. 6.3. Alcance Técnico: Reproducibilidad e Infraestructura . . . . .   | 14        |
| 1. 6.4. Supuestos y Limitaciones . . . . .                              | 14        |
| <b>2. Marco teórico</b>                                                 | <b>17</b> |
| 2. 1. Formulación matemática del MDRP . . . . .                         | 17        |
| 2. 1.1. Conjuntos y entidades . . . . .                                 | 17        |
| 2. 1.2. Suposiciones estructurales . . . . .                            | 18        |
| 2. 1.3. Modelo de compensación . . . . .                                | 19        |
| 2. 1.4. Métricas de desempeño . . . . .                                 | 19        |
| 2. 1.5. Variantes del entorno operativo . . . . .                       | 20        |
| 2. 1.6. Clasificación del MDRP en la literatura . . . . .               | 20        |
| 2. 2. Heurística de horizonte rodante . . . . .                         | 21        |
| 2. 2.1. Tamaño objetivo de agrupamiento . . . . .                       | 21        |
| 2. 2.2. Generación de agrupamientos . . . . .                           | 22        |
| 2. 2.3. Esquema de prioridad . . . . .                                  | 22        |
| 2. 2.4. Modelo de asignación lineal . . . . .                           | 23        |
| 2. 2.5. Estrategia de compromiso en dos etapas . . . . .                | 24        |
| 2. 3. Enrutamiento sobre grafos viales . . . . .                        | 25        |
| 2. 3.1. OpenStreetMap como modelo de datos geográficos . . . . .        | 25        |
| 2. 3.2. Contraction Hierarchies . . . . .                               | 26        |
| 2. 3.3. OSRM: motor de enrutamiento . . . . .                           | 26        |
| 2. 4. Generación estocástica de demanda y métricas . . . . .            | 27        |
| 2. 4.1. Proceso de Poisson no homogéneo . . . . .                       | 27        |
| 2. 4.2. Función de intensidad bimodal . . . . .                         | 28        |
| 2. 4.3. Métricas de desempeño: marco de evaluación . . . . .            | 29        |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>3. Desarrollo</b>                                                           | <b>31</b> |
| 3. 1. Arquitectura del sistema . . . . .                                       | 31        |
| 3. 1.1. Capa de enrutamiento: OSRM contenerizado . . . . .                     | 31        |
| 3. 1.2. Capa de simulación: estructura de módulos Python . . . . .             | 35        |
| 3. 1.3. Capa de análisis: pipeline de resultados . . . . .                     | 36        |
| 3. 2. Generación de instancias sintéticas . . . . .                            | 38        |
| 3. 2.1. Generación temporal de órdenes: proceso Poisson no homogéneo . . . . . | 38        |
| 3. 2.2. Tiempo de preparación . . . . .                                        | 38        |
| 3. 2.3. Distribución geoespacial . . . . .                                     | 38        |
| 3. 2.4. Dimensionamiento automático de flota . . . . .                         | 39        |
| 3. 3. Motor de simulación . . . . .                                            | 40        |
| 3. 3.1. Ciclo de simulación por épocas . . . . .                               | 40        |
| 3. 3.2. Modelos de datos: órdenes, couriers y restaurantes . . . . .           | 41        |
| 3. 3.3. Máquina de estados de una orden . . . . .                              | 42        |
| 3. 3.4. Integración con OSRM . . . . .                                         | 42        |
| 3. 4. Política FCFS (baseline) . . . . .                                       | 43        |
| 3. 4.1. Lógica de asignación . . . . .                                         | 44        |
| 3. 4.2. Características y limitaciones . . . . .                               | 45        |
| 3. 5. Política Rolling Horizon . . . . .                                       | 46        |
| 3. 5.1. Cálculo del tamaño objetivo de agrupamiento . . . . .                  | 46        |
| 3. 5.2. Generación de agrupamientos por restaurante . . . . .                  | 46        |
| 3. 5.3. Clasificación por prioridad y asignación bipartita . . . . .           | 47        |
| 3. 5.4. Compromiso en dos etapas . . . . .                                     | 48        |
| 3. 5.5. Resumen del flujo RH por época . . . . .                               | 49        |
| 3. 6. Diseño experimental . . . . .                                            | 49        |
| 3. 6.1. Variable de tratamiento: saturación de flota . . . . .                 | 51        |
| 3. 6.2. Escenarios . . . . .                                                   | 51        |
| 3. 6.3. Comparación pareada . . . . .                                          | 51        |
| 3. 6.4. Replicación y análisis de sensibilidad . . . . .                       | 51        |
| 3. 6.5. Almacenamiento de resultados . . . . .                                 | 52        |
| 3. 7. Métricas de evaluación . . . . .                                         | 52        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| 3. 7.1. Calidad de servicio . . . . .                       | 53        |
| 3. 7.2. Cobertura . . . . .                                 | 53        |
| 3. 7.3. Eficiencia operativa . . . . .                      | 53        |
| 3. 7.4. Eficiencia de agrupamiento . . . . .                | 54        |
| 3. 7.5. Costo . . . . .                                     | 54        |
| 3. 7.6. Resumen de indicadores . . . . .                    | 54        |
| <b>4. Resultados</b>                                        | <b>56</b> |
| 4. 1. Configuración del experimento . . . . .               | 56        |
| 4. 2. Calidad de servicio . . . . .                         | 56        |
| 4. 3. Validación estadística . . . . .                      | 56        |
| 4. 3.1. Interpretación de resultados estadísticos . . . . . | 56        |
| 4. 3.2. Verificación de la hipótesis . . . . .              | 56        |
| 4. 3.3. Limitaciones de los resultados . . . . .            | 56        |
| <b>5. Conclusiones</b>                                      | <b>57</b> |
| <b>Bibliografía</b>                                         | <b>57</b> |

# Índice de figuras

- 2.1. Comparación del modelo base frente al modelo con variantes (preposicionamiento y actualización local de asignaciones). . . . . 20
- 3.1. Arquitectura de tres capas del entorno de simulacion MDRP-BCS. . . . . 32
- 3.2. Flujo de la política Rolling Horizon en cada época de asignación. . . . . 50

# Índice de tablas

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| 3.1. Módulos del simulador y su responsabilidad. . . . .                                        | 36 |
| 3.2. Parámetros de configuración del simulador ( <code>config.py</code> ). . . . .              | 37 |
| 3.3. Escenarios experimentales de saturación ( $ \mathcal{O}  \approx 1\,038$ pedidos). . . . . | 51 |
| 3.4. Indicadores clave de desempeño (KPIs) evaluados. . . . .                                   | 55 |



# Capítulo 1

## Introducción

### 1. 1. Antecedentes

#### 1. 1.1. De TSP a VRP

El Problema del Vendedor Viajero (TSP, por sus siglas en inglés) es uno de los problemas de optimización combinatoria más estudiados y fundamentales en la investigación de operaciones. En su forma más simple, el problema busca determinar el recorrido más corto posible que visita un conjunto de ciudades exactamente una vez, comenzando y terminando en la misma ciudad.

A pesar de su sencilla definición, el TSP es NP-duro, ya que el tiempo requerido para encontrar una solución óptima crece factorialmente con el número de ciudades, haciendo que las instancias de gran tamaño sean computacionalmente intratables para métodos exactos [1].

El TSP sirve como pilar para una clase más amplia y práctica de problemas de logística: el Problema de Enrutamiento de Vehículos (VRP). La transición conceptual del TSP al VRP ocurrió con el trabajo seminal de Dantzig and Ramser [2], quienes formularon "The Truck Dispatching Problem". Este trabajo generalizó el TSP al considerar una flota de vehículos con capacidad limitada, con el objetivo de servir a un conjunto de clientes desde un depósito central. Esta generalización marcó el nacimiento del VRP, que busca minimizar el costo total de las rutas (usualmente en términos de distancia o tiempo) para una flota de vehículos que atiende a un conjunto de clientes con demandas conocidas.

Al igual que el TSP, el VRP es NP-duro [1], pero su estructura es significativamente más rica y compleja. La complejidad no solo reside en encontrar las secuencias óptimas de visitas

(como en el TSP), sino también en asignar clientes a vehículos y rutas de manera eficiente [3]. A lo largo de cinco décadas, el VRP ha evolucionado para incorporar una gran variedad de restricciones del mundo real, dando lugar a los llamados Rich VRPs” [3]. Estas extensiones incluyen, entre otras:

### 1. 1.2. VRP clásico y extensiones con restricciones

- **Capacidades de los vehículos (CVRP):** Donde cada vehículo tiene una capacidad máxima de carga que no puede ser excedida. El CVRP introduce la decisión adicional de no solo determinar la secuencia de visitas, sino también decidir cuánto llevar en cada viaje, lo que aumenta significativamente la complejidad computacional del problema. Cada vehículo debe partir desde un depósito central y retornar a él, respetando siempre la restricción de capacidad [3].
- **Ventanas de tiempo (VRPTW):** Donde cada cliente debe ser atendido dentro de un intervalo de tiempo específico. Las ventanas de tiempo añaden complejidad porque no solo se optimiza la distancia de las rutas, sino también el tiempo, considerando que los vehículos pueden llegar temprano (y esperar con costo de tiempo de espera) o llegar tarde (incurriendo en penalizaciones). Esta restricción introduce la dimensión temporal explícita al problema de enrutamiento [3].
- **Múltiples depósitos (MDVRP):** Donde los vehículos pueden iniciar y terminar sus rutas desde diferentes ubicaciones, en lugar de un único depósito central. Esto introduce complejidad adicional porque el algoritmo debe decidir no solo las rutas de visita, sino también asignar qué vehículos operan desde qué depósitos. El MDVRP es relevante en contextos logísticos donde múltiples centros de distribución sirven diferentes regiones geográficas [3].

Más allá de estas extensiones fundamentales, la literatura identificada en Laporte [4] documenta otras variantes relevantes, como problemas con *pickup* y *delivery* (recogida y entrega en diferentes ubicaciones), problemas con *splits* de demanda (dividir la demanda de un cliente entre múltiples vehículos) y problemas con tiempos de servicio variables. Cada una de estas extensiones añade dimensiones de realismo operacional, pero también incrementa considerablemente la dificultad de encontrar soluciones óptimas.

La progresión desde el VRP clásico hacia variantes más complejas establece el contexto para entender el Problema de Enrutamiento de Entrega de Comida (MDRP). El MDRP no es simplemente una combinación de estas restricciones estáticas; es un problema fundamentalmente dinámico en el que nuevas órdenes arriban continuamente durante el día operativo, requiriendo que las decisiones de enrutamiento se reoptimicen periódicamente. Este cambio de paradigma de problemas estáticos a dinámicos, junto con la necesidad de considerar redes viales reales, sitúa la motivación y relevancia del presente trabajo.

### 1. 1.3. Última milla y contexto urbano

La última milla de entrega—el segmento final desde un centro de distribución o punto de recogida hasta el domicilio del cliente—representa el componente más costoso, complejo y crítico en logística urbana. Jazemi *et al.* (2023) documentan que la última milla puede representar entre el 30 % y el 60 % del costo total de una operación de envío, dependiendo de la densidad urbana, la dispersión geográfica de clientes y la volatilidad de la demanda [5].

En el contexto de entrega de comida rápida, la última milla presenta desafíos específicos adicionales: (1) alta urgencia—los clientes esperan entregas en ventanas de 30-60 minutos desde la orden, significativamente más restrictivas que en logística general; (2) perecedibilidad—la comida debe mantener temperatura, frescura y presentación, implicando rutas directas sin desviaciones innecesarias; (3) demanda dinámica e impredecible—los flujos de órdenes no son uniformes sino que presentan picos pronunciados (horas de almuerzo, cena), requiriendo reoptimización continua de asignaciones.

Estas características transforman la última milla de comida en un problema distinto del VRP clásico: es inherentemente dinámico, con ventanas de tiempo estrictas y demanda estocástica. Aunque la literatura de enrutamiento dinámico [6, 7] y las estrategias de anticipación [8] han abordado componentes de este desafío, la entrega de comida presenta particularidades únicas que requieren una integración específica de estos conceptos, como se explora en la siguiente sección.

### 1. 1.4. Enrutamiento dinámico: DVRP y DARP

Mientras que el VRP clásico y sus extensiones (CVRP, VRPTW, MDVRP) asumen que todas las demandas de clientes son conocidas *ex-ante* (antes de la ejecución), la realidad operacional de los sistemas de entrega urbana es fundamentalmente distinta: nuevas órdenes llegan continuamente durante el día, y las decisiones de despacho deben reoptimizarse en tiempo real. Este cambio de paradigma de lo estático a lo dinámico define una nueva clase de problemas de enrutamiento que requieren políticas de despacho adaptativas.

### 1. 1.5. DVRP: Definición y características fundamentales

Larsen (2000) proporciona la taxonomía fundamental del Problema de Enrutamiento de Vehículos Dinámico (DVRP), definiéndolo como una clase de problemas donde, en contraste con el VRP estático, “nuevas solicitudes de clientes arriban en tiempo real durante la operación, y el conjunto de clientes a atender no es conocido completamente en el inicio” [9]. Formalmente, el DVRP se caracteriza por:

- **Llegada estocástica de solicitudes:** Las órdenes no existe como conjunto fijo, sino que arriban según un proceso estocástico.
- **Reoptimización periódica:** El plan de enrutamiento no es estático sino que debe recalcularse continuamente. Las decisiones previas (repartidores ya en ruta) no pueden revertirse completamente, pero las nuevas decisiones pueden considerar órdenes recién llegadas.
- **Incertidumbre temporal:** No solo la demanda es incierta, sino también los tiempos de viaje pueden variar por congestión u otros factores, lo que crea desviaciones entre el plan y la ejecución.
- **Disyuntiva decisión-información:** Cada política de despacho enfrenta un dilema: decidir inmediatamente (riesgo de decisiones subóptimas) vs. esperar más información (riesgo de incumplir criterios de servicio). Pillac et al. (2013) profundizan en este aspecto, clasificando los problemas según su grado de dinamismo y la calidad de la información disponible [10].

Esta caracterización es central para entender la última milla de entrega: cada vez que un cliente coloca una orden en una plataforma de entrega de comida (Grubhub, Uber Eats, Door-Dash), el sistema de despacho enfrenta una decisión urgente: asignar esa orden a un repartidor para que sea recogida y entregada. La decisión no puede posponerse indefinidamente (los clientes esperan entregas en 30-60 minutos), pero tampoco puede hacerse de forma completamente reactiva sin consideración del contexto futuro.

### 1. 1.6. **DARP: Del transporte de pasajeros a la entrega de bienes**

Un problema históricamente relacionado pero conceptualmente distinto es el Dial-a-Ride Problem (DARP). Las raíces del DARP se encuentran en Psaraftis (1980), quien formuló el primer modelo dinámico de dial-a-ride: en sistemas de transporte compartido (típicamente para pasajeros), usuarios solicitan recogida en una ubicación y entrega en otra ubicación diferente, y ambas solicitudes llegan en tiempo real. Psaraftis (1980) abordó la pregunta central: ¿cómo asignar dinámicamente los pasajeros a vehículos y secuenciar sus viajes de forma óptima bajo políticas de despacho inmediato? [6]

Cordeau y Laporte (2007) extendieron significativamente este trabajo, formalizando matemáticamente el Dial-a-Ride Problem y desarrollando heurísticas de solución prácticas [7]. Su contribución fue crítica para la investigación en enrutamiento dinámico. Definieron explícitamente las restricciones estructurales del DARP:

- **Restricción de capacidad:** Cada vehículo tiene límite de pasajeros simultáneos.
- **Restricción de secuencia temporal:** Un pasajero debe ser recogido antes de ser entregado, con límite temporal máximo entre recogida y entrega.
- **Ventanas de tiempo:** Tanto recogida como entrega pueden tener ventanas horarias de disponibilidad.
- **Criterios de optimización:** Minimización de distancia total o tiempo total de espera, frecuentemente con penalizaciones por incumplimiento de compromisos de tiempo.

Aunque el DARP fue originalmente formulado para transporte de pasajeros, su estructura conceptual es directamente transferible a problemas de entrega de bienes. De hecho, la estructura

matemática es análoga: en lugar de “recoger un pasajero en ubicación A y entregar en ubicación B”, se tiene “recoger una orden en restaurante A y entregar en domicilio B”. Sin embargo, existen diferencias operacionales críticas:

- En transporte de pasajeros, la restricción de secuencia es rígida (recogida antes de entrega), mientras que en entrega de comida existe más flexibilidad: múltiples órdenes de diferentes restaurantes pueden incluirse en una sola ruta.
- En transporte de pasajeros, el tiempo entre recogida y entrega es determinístico (depende de la ruta), mientras que en comida existe heterogeneidad: tiempo de preparación varía por restaurante, impactando cuando la orden está disponible para recogida.
- En transporte de pasajeros, la experiencia del cliente es inmediata (espera observable), mientras que en comida, el cliente está dispuesto a esperar mientras el pedido se prepara, reduciendo la presión sobre latencia de decisión de despacho.

### 1. 1.7. El valor de la información futura y el enfoque de Horizonte Rodante

A pesar de la diferencia entre DVRP y DARP, ambas clases de problemas comparten una pregunta fundamental: ¿Deben las decisiones ser reactivas (FCFS) o pueden ser anticipatorias?

Ichoua et al. (2006) abordaron directamente esta pregunta en el contexto de *dial-a-ride dinámico*. Compararon políticas de despacho que desconocen solicitudes futuras (puramente reactivas) contra políticas que tienen acceso a información sobre solicitudes que llegarán en los próximos 20-30 minutos. Su hallazgo fue significativo: en escenarios críticos con elevada tasa de llegada de solicitudes, políticas anticipatorias pueden lograr mejoras de hasta el 29 % en la reducción de clientes no servidos y entre 4-16 % en tiempo total y retrasos, demostrando que la anticipación moderada proporciona ganancias sustanciales sin requerir información perfecta sobre el futuro distante [8].

Esta idea fundamental conecta DVRP/DARP con el concepto de horizonte rodante (*rolling horizon*): en lugar de una asignación inmediata (FCFS), el sistema puede esperar un breve período (el horizonte  $\Delta$ , típicamente 10-30 minutos), acumular un lote de solicitudes cuyo arribo es conocido con certeza relativa, optimizar globalmente ese lote resolviendo un VRP estático

(el “matching” óptimo), y luego ejecutar la primera etapa de esa solución optimizada. En la siguiente época, repite el proceso.

### **1. 1.8. MDRP y políticas de despacho**

El Problema de Enrutamiento de Entrega de Comida (MDRP) fue formalmente introducido por Reyes et al. (2018) como una nueva clase de problema de enrutamiento dinámico que, aunque comparte características fundamentales con DVRP y DARP, añade dimensiones específicas de la entrega de alimentos que lo distinguen conceptualmente y requieren políticas de despacho especializadas.

### **1. 1.9. Características compartidas con DVRP y DARP**

Como todo problema DVRP, el MDRP hereda la característica de dinamicidad: nuevas órdenes arriban continuamente en tiempo real, y el conjunto de demanda no es conocido *ex-ante*. Como problema similar a DARP, el MDRP hereda la estructura de recogida-entrega: una orden debe ser recogida en un restaurante (fuente) y entregada en un domicilio (destino), con restricciones de ventanas de tiempo y capacidad de vehículos. Estas características mencionadas crean el contexto operacional básico.

Sin embargo, Reyes *et al.* (2018) evidenciaron que estas dos extensiones no capturan plenamente la realidad de la entrega de comida en ultima milla. El MDRP fue definido para llenar esta brecha, formulando un problema que integra cinco características estructurales que no aparecen de forma acoplada en la literatura previa:

### **1. 1.10. Definición y características del MDRP**

Reyes *et al.* (2018) definen el MDRP como una clase de problema de recolección y entrega dinámico (DPDP) caracterizado por una urgencia extrema y menores oportunidades de consolidación en comparación con la logística tradicional. Sus características distintivas son:

1. **Múltiples puntos de recolección (Restaurantes):** A diferencia del VRP clásico con un depósito central, el MDRP opera con múltiples restaurantes dispersos, cada uno actuando como un punto de recolección independiente con ubicación fija.

2. **Tiempos de preparación:** Las órdenes no están listas inmediatamente al ser colocadas. Reyes et al. (2018) introducen el concepto de *ready time* (tiempo de preparación) como una restricción dura: el courier no puede recoger la orden antes de que la comida esté lista. Esto crea una holgura de tiempo natural pero reduce la flexibilidad del enrutamiento.
3. **Alta urgencia y Dinamismo:** La mayoría de las solicitudes se revelan durante la operación (dinamismo) y deben completarse en ventanas de tiempo muy cortas, típicamente menos de una hora (urgencia). Reyes et al. (2018) señalan que esta combinación de dinamismo y urgencia es el desafío central del MDRP.
4. **Agrupamiento dinámico (*bundling*):** Para mejorar la eficiencia, el MDRP permite combinar múltiples órdenes de un mismo restaurante en una sola ruta, creando agrupamientos (referidos en la literatura inglesa como "*bundles*"). Aunque el modelo teórico de Reyes et al. (2018) asume capacidad ilimitada para estos agrupamientos, en la implementación práctica de esta tesis se considera la capacidad física limitada de los vehículos (motocicletas/bicicletas) como una restricción operativa real.

### 1. 1.11. Enrutamiento realista en redes viales

La validación efectiva de políticas de despacho dinámico depende críticamente de la calidad de los datos de enrutamiento utilizados en la simulación. La literatura actual presenta una división instrumental: por un lado, las plataformas comerciales (Google Maps, Mapbox) ofrecen máxima precisión en tiempos de viaje, pero su naturaleza de "caja negra", sumada a estructuras de costos restrictivas, impide la auditoría de los algoritmos y compromete la reproducibilidad exacta de los experimentos científicos [11]. Por otro lado, la investigación académica tradicional frecuentemente opta por simplificaciones euclidianas que, aunque gratuitas y totalmente reproducibles, carecen de fidelidad operativa.

Estudios empíricos recientes demuestran que estas aproximaciones geométricas subestiman sistemáticamente los tiempos de viaje, introduciendo errores de magnitud crítica para la operación. Boyacı et al. [12], tras analizar 96 instancias en 12 ciudades reales, determinaron que las rutas efectivas son entre un 17 % y un 40 % más largas que la distancia lineal. En términos operativos, esta desviación implica que un trayecto estimado geométricamente en 20 minutos podría requerir en realidad casi 30 minutos, provocando el incumplimiento sistemático de las



promesas de entrega. En este sentido, Jazemi et al. [5] enfatizan que las estrictas ventanas de tiempo en la logística de última milla requieren estimaciones precisas de arribo (ETA) que no pueden garantizarse con métricas simples. Bajo estas condiciones, Boyacı et al. [12] advierten que el uso de distancias lineales resulta frecuentemente en la generación de rutas operativamente infactibles.

## 1. 2. Planteamiento del problema

### 1. 2.1. La ineficacia de las aproximaciones tradicionales

El problema de enrutamiento para entrega de comida (MDRP) en ciudades medianas enfrenta una disyuntiva crítica. Por un lado, las herramientas comerciales de enrutamiento garantizan precisión geográfica, pero imponen costos de licenciamiento prohibitivos para las empresas locales emergentes. Por otro lado, las aproximaciones académicas tradicionales, que suelen utilizar distancias euclidianas para reducir costos computacionales, sacrifican el realismo operativo. Boyacı et al. [12] cuantificaron este impacto, demostrando que las distancias lineales subestiman los trayectos reales entre un 17 % y un 40 %, invalidando las estimaciones de tiempos de entrega (ETA) y degradando la calidad del servicio.

### 1. 2.2. La hostilidad topológica de La Paz, B.C.S.

Esta problemática de subestimación se agudiza drásticamente en el contexto local de La Paz, B.C.S., cuya topología dista de ser un plano euclidiano ideal o una retícula urbana densa como las grandes metrópolis. La ciudad presenta restricciones geográficas severas:

- **Barreras Naturales:** La ensenada de La Paz obliga a realizar rodeos extensos (en forma de u) para trayectos que linealmente parecen cortos a través del agua.
- **Estructura Vial Jerárquica:** La dependencia de pocas arterias diagonales estructurantes (como el Blvd. Forjadores o Pino Pallas) crea cuellos de botella que los modelos euclidianos ignoran.

Un cálculo de ruta basado en distancia lineal en este entorno no solo es inexacto, sino que frecuentemente resulta en rutas infactibles, alterando drásticamente la viabilidad operativa en

las entregas de comida de última milla.

### 1. 2.3. La brecha en la eficiencia algorítmica

Si bien Reyes et al. [13] demostraron que las políticas de despacho basadas en horizonte rodante (Rolling Horizon) pueden alcanzar eficiencias cercanas al 96 % respecto al óptimo teórico, su validación se restringió a instancias sintéticas de grandes urbes con alta conectividad y, crucialmente, empleó distancias euclidianas.

Actualmente, se desconoce el grado de degradación que sufren estas políticas de optimización al ser trasladadas a un entorno de baja conectividad y alta penalización por error como La Paz. La falta de validación de estas estrategias en topologías hostiles deja a los desarrolladores y empresas locales sin una base científica para implementar sistemas de logística automatizada eficientes, perpetuando la dependencia de la gestión manual empírica y limitando su competitividad frente a plataformas transnacionales.

## 1. 3. Objetivos

### 1. 3.1. Objetivo general

Evaluar la eficiencia operativa de la política de despacho *Rolling Horizon* frente a una estrategia reactiva (FCFS) en el problema MDRP, mediante el desarrollo de un entorno de simulación reproducible que integre restricciones topológicas reales de la ciudad de La Paz, B.C.S. y patrones de demanda estocásticos.

### 1. 3.2. Objetivos específicos

- Generar un conjunto de instancias sintéticas calibradas que proyecten los patrones temporales de demanda de la literatura (LaDe) sobre la estructura demográfica real de La Paz, garantizando la validez geoespacial de los puntos de entrega y recolección.
- Integrar un motor de simulación de eventos discretos con la infraestructura de enrutamiento OSRM sobre cartografía OpenStreetMap, validando la conectividad del grafo vial y la precisión de los tiempos de viaje frente a distancias euclidianas.

- Implementar computacionalmente la heurística de Horizonte Rodante (*Rolling Horizon*) con capacidad de agrupamiento dinámico (*bundling*) y la política de control *First-Come, First-Served*, instrumentando la recolección de métricas de desempeño.
- Cuantificar las diferencias de desempeño entre ambas políticas mediante análisis estadístico de los KPIs críticos: tiempos *Click-to-Door*, latencia *Ready-to-Pickup* y eficiencia de la flota, determinando la significancia de las mejoras observadas.

## 1. 4. Hipótesis

La implementación de una política de despacho anticipatoria (Horizonte Rodante), alimentada con una matriz de costos de viaje reales (OSRM) sobre la topología de La Paz, superará significativamente en eficiencia operativa "órdenes por hora" nivel de servicio "*Ready-to-Door*" a la estrategia reactiva estándar (FCFS con estimación euclidiana), demostrando la viabilidad de soluciones de optimización logística de alto rendimiento basadas exclusivamente en software de código abierto.

## 1. 5. Justificación

La creciente demanda de servicios de entrega bajo demanda *on-demand delivery* ha transformado la logística urbana, imponiendo desafíos que los métodos exactos tradicionales no pueden resolver en tiempo real debido a su complejidad exponencial. Si bien el modelo de *Rolling Horizon* [13] ofrece una solución teórica prometedora, su aplicabilidad práctica en ciudades latinoamericanas de escala media se ve limitada por la carencia de validación en topologías viales restrictivas y la dependencia de costos prohibitivos de software propietario. La presente investigación se justifica bajo las siguientes dimensiones:

### 1. 5.1. Relevancia Científica

Existe una disonancia documentada entre los modelos teóricos de enrutamiento, que asumen espacios euclidianos o redes ideales, y la realidad operativa de la última milla. Figliozzi [14] advierte que las aproximaciones lineales distorsionan significativamente los costos de viaje. El

presente trabajo busca cerrar esa brecha al validar si las eficiencias reportadas en la literatura (cercanas al óptimo) se sostienen al introducir las restricciones de la topología de La Paz, B.C.S., aportando nueva evidencia sobre la robustez de las heurísticas de horizonte rodante.

### 1. 5.2. Aporte Tecnológico y Ciencia Abierta

Este proyecto contribuye con una arquitectura de software diseñada bajo principios de Ciencia Abierta. Se integra el motor de enrutamiento OSRM desplegando su **imagen oficial en Docker**, lo que permite sustituir las distancias euclidianas por tiempos de viaje reales sobre la red vial, asegurando la consistencia del entorno de cálculo [15]. Esta integración desacopla la complejidad de la infraestructura de enrutamiento de la lógica de simulación, facilitando que la comunidad académica replique los experimentos sin enfrentar problemas de configuración.

### 1. 5.3. Impacto Económico y Social

Para las PyMES y startups locales en Baja California Sur, la dependencia de APIs comerciales (como Google Maps) representa una barrera de entrada económica. Al demostrar la viabilidad técnica de optimizar rutas utilizando exclusivamente herramientas *Open Source* (OSRM), esta tesis ofrece una alternativa de "Soberanía Tecnológica" que reduce los costos operativos. Esto democratiza el acceso a logística de alto nivel, permitiendo que empresas locales compitan en eficiencia contra plataformas transnacionales, fomentando así la economía digital regional.

### 1. 5.4. Delimitación frente a métodos de Aprendizaje Profundo

Aunque existen métodos avanzados de Inteligencia Artificial (como Aprendizaje Profundo), estos requieren grandes cantidades de datos históricos para funcionar. En ciudades como La Paz, donde no existen registros previos de rutas y pedidos, estos modelos enfrentan el problema de "arranque en frío": no pueden operar sin entrenamiento previo.

Por ello, esta tesis utiliza *Rolling Horizon*, un algoritmo que no necesita datos históricos y funciona desde el primer día de operación. Además, al ser computacionalmente ligero, permite a las empresas locales operar con servidores económicos. Este sistema servirá, a su vez, para generar los datos necesarios que permitirán entrenar modelos de IA en el futuro.

## 1. 6. Alcance y limitaciones

El presente trabajo de investigación se enfoca en el desarrollo, implementación y evaluación de un entorno de simulación para el problema MDRP. A continuación se detallan las fronteras funcionales, geográficas y técnicas del proyecto, así como las restricciones operativas asumidas.

### 1. 6.1. Alcance Geográfico y de Datos

#### Infraestructura Vial (La Paz, B.C.S.)

Geográficamente nos limitamos estrictamente a la zona urbana de La Paz, Baja California Sur. Se implementa una instancia local del servidor de enrutamiento **OSRM v5.26.0** que opera sobre `localhost`, eliminando la dependencia de conectividad externa a internet y garantizando latencia mínima. Este servidor procesa el extracto en formato `.pbk` de OpenStreetMap correspondiente a la ciudad de La Paz.

#### Instancias de Demanda y Patrones Temporales

El alcance se limita a la generación de **instancias sintéticas calibradas**, construidas bajo los siguientes parámetros:

- **Volumen:** Escenarios de aproximadamente 1,000 órdenes distribuidas en una ventana operativa de 11 horas (08:00 - 19:00).
- **Patrones Estocásticos:** Distribución de pedidos basada en los picos de demanda del dataset de referencia *LaDe* [16], adaptados a la zona horaria local.

### 1. 6.2. Alcance Funcional y Algorítmico

El sistema compara exclusivamente dos políticas de despacho::

1. **Política Base *First-Come-First-Served* (FCFS):** Asignación reactiva inmediata basada en minimización de distancia al restaurante, sin consolidación de órdenes.
2. **Política Propuesta (*Rolling Horizon*):** Implementación de la heurística de Reyes et al. [13] con un horizonte de visión configurable (10-20 minutos) y capacidad de agrupamiento (*bundling*) dinámico de hasta 4 órdenes por repartidor.

3. **Escenario de Control (Aislamiento de Variables):** Se contempla la ejecución de un escenario de control (FCFS sobre OSRM) para aislar la contribución del algoritmo de optimización frente a la mejora atribuible puramente a la precisión del mapa, permitiendo un análisis independiente de ambas variables.

La simulación se ejecuta bajo un esquema de **pasos de tiempo fijos**, una simulación donde el estado del sistema se evalúa y actualiza en intervalos regulares de reoptimización ( $f = 5$  min). Este enfoque se alinea con la naturaleza periódica del algoritmo *Rolling Horizon* propuesto por Reyes et al. [13], permitiendo capturar la estocasticidad en la llegada de órdenes mientras se mantiene la sincronización con los ciclos de decisión de la flota.

### 1. 6.3. Alcance Técnico: Reproducibilidad e Infraestructura

El proyecto entrega software reproducible, diseñado para facilitar la auditoría y la extensión futura. El alcance técnico garantiza:

- **Infraestructura de Enrutamiento Contenerizada:** El motor de enrutamiento OSRM se despliega mediante **Docker** (imagen `osrm/osrm-backend`), configurado con el algoritmo CH (Contraction Hierarchies). Esto asegura que el entorno de cálculo de rutas sea idéntico en cualquier máquina anfitriona, eliminando discrepancias por versiones de librerías del sistema operativo.
- **Independencia de APIs:** El sistema opera de forma autónoma (*offline*) sobre `localhost`, sin consumo de créditos de Google Maps o Mapbox, eliminando barreras económicas para la replicación de experimentos.
- **Requisitos Computacionales:** El entorno está diseñado para ejecutarse en hardware de consumo estándar (CPU x86/ARM, 8GB RAM), democratizando el acceso a herramientas de optimización logística avanzada.

### 1. 6.4. Supuestos y Limitaciones

Las conclusiones del estudio deben interpretarse bajo las siguientes restricciones metodológicas y simplificaciones del modelo:

## Modelo de Tráfico Estático

El cálculo de tiempos de viaje asume velocidades promedio constantes por tipo de vía (perfil `car.lua` ajustado). **Limitación:** No se modela la congestión vehicular dinámica en tiempo real (accidentes o cierres viales temporales). Esto implica que las métricas de tiempo de entrega (*Click-to-Door*) representan un caso optimista respecto a la realidad operativa con tráfico saturado.

## Simplificación de la Logística de Última Milla

El modelo asume tiempos de servicio deterministas divididos en dos componentes:

- **Tiempo de Recolección (*Pickup*):** Se aplica una penalización fija de  $T_{pickup} = 4$  min por visita a cada restaurante asignado por visitar.
- **Tiempo de Entrega (*Drop-off*):** Se añade un tiempo de servicio de  $T_{drop} = 4$  min por cada orden entregada al cliente.
- **Ausencia de Variabilidad Estocástica:** No se modelan retrasos aleatorios por búsqueda de estacionamiento, acceso a edificios complejos o clientes ausentes; los tiempos son constantes.

## Comportamiento de la Flota

- **Velocidad Constante:** Los repartidores viajan a la velocidad constante estimada por OSRM, sin desviaciones por comportamiento humano.
- **Sin Retorno a Base:** Al finalizar una entrega, los repartidores permanecen estáticos en la ubicación del último cliente hasta recibir una nueva asignación. No se modelan estrategias de reposicionamiento o retorno a zonas de alta demanda.
- **Energía y combustible** No se modelan paradas técnicas por recarga de combustible o batería durante el turno operativo.

## Limitaciones Algorítmicas

- **Heurística de Asignación Jerárquica:** Siguiendo la metodología de Reyes et al. [13], se adopta una estrategia secuencial que prioriza órdenes críticas (Grupos de prioridad).

- **Compromiso en Dos Etapas:** La lógica de "Compromiso Parcial" permite redirigir a un repartidor en ruta, lo cual es una simplificación operativa que asume obediencia total y comunicación instantánea sin fricción.

### **Escenario Nominal de Operación**

La simulación asume condiciones de flujo ideal. Quedan fuera del alcance:

- Cancelaciones de órdenes por parte de clientes o restaurantes.
- Rechazo de asignaciones por parte de los repartidores.
- Fallas mecánicas de la flota o gestión de efectivo.
- *Multihoming* (repartidores trabajando para múltiples apps simultáneamente).
- **Homogeneidad de carga:** Se asume que todas las órdenes poseen dimensiones estándar compatibles con la mochila del repartidor, ignorando restricciones de volumen físico o peso excesivo.

### **Generalización de Patrones de Consumo**

Aunque las distribuciones estadísticas (llegadas Poisson) se calibran con literatura vigente [16], se asume que los patrones de comportamiento de consumo son universales y transferibles al contexto de La Paz.



# Capítulo 2

## Marco teórico

Este capítulo presenta los fundamentos teóricos que sustentan el diseño del simulador y las políticas de despacho implementadas en el Capítulo 3. Se formaliza el Meal Delivery Routing Problem (MDRP), se describe la heurística de horizonte rodante, se revisan las técnicas de enrutamiento sobre grafos viales y se introduce el modelo estocástico de generación de demanda.

### 2. 1. Formulación matemática del MDRP

El **Meal Delivery Routing Problem** (MDRP), introducido por Reyes et al. [13], formaliza la clase de problemas de ruteo dinámico que surgen en las operaciones de entrega de comida bajo demanda. Esta sección reproduce la notación y las suposiciones estructurales del modelo original, que constituyen la base formal del simulador desarrollado en este trabajo.

#### 2. 1.1. Conjuntos y entidades

El MDRP se define sobre tres conjuntos fundamentales:

- $\mathcal{R}$ : conjunto de **restaurantes**. Cada restaurante  $r \in \mathcal{R}$  posee una ubicación fija  $\ell_r$ .
- $\mathcal{O}$ : conjunto de **órdenes** (pedidos). Cada orden  $o \in \mathcal{O}$  se caracteriza por:
  - restaurante de origen  $r_o \in \mathcal{R}$ ,
  - instante de colocación (*placement time*)  $a_o$ ,
  - instante de disponibilidad (*ready time*)  $e_o \geq a_o$ ,

- ubicación de entrega (*drop-off*)  $\ell_o$ .

La información de una orden  $o$  se revela al sistema únicamente en el instante  $a_o$  (llegada dinámica).

■  $\mathcal{C}$ : conjunto de **repartidores** (couriers). Cada courier  $c \in \mathcal{C}$  posee:

- instante de inicio de turno (*on-time*)  $e_c$ ,
- ubicación inicial  $\ell_c$  en el instante  $e_c$ ,
- instante de fin de turno (*off-time*)  $l_c > e_c$ .

A diferencia de las órdenes, toda la información sobre restaurantes y couriers se conoce *a priori*.

El MDRP consiste en determinar rutas factibles para los couriers que completen la recolección y entrega de las órdenes, optimizando una o varias métricas de desempeño.

## 2. 1.2. Suposiciones estructurales

Las siguientes suposiciones, tomadas directamente de Reyes et al. [13], definen la factibilidad de las soluciones:

1. **Agrupamiento (*bundling*):** Órdenes del mismo restaurante pueden combinarse en un *agrupamiento* (bundle) con múltiples puntos de entrega. El *ready time* del agrupamiento es el máximo *ready time* de sus órdenes constituyentes.
2. **Tiempos de viaje:** El tiempo de viaje entre cualquier par de ubicaciones se asume invariante en el tiempo.
3. **Tiempos de servicio:** Se definen dos tiempos de servicio constantes:  $s^r$  para la recolección en un restaurante (independiente del número de órdenes) y  $s^o$  para la entrega al cliente.
4. **Restricción de turno:** Un courier no puede iniciar nuevas recolecciones después de su *off-time*  $l_c$ , pero puede completar entregas pendientes.
5. **Obediencia determinista:** En el marco determinista del modelo, los couriers aceptan todas las instrucciones y esperan nuevas asignaciones en su última ubicación conocida.

### 2. 1.3. Modelo de compensación

La remuneración de los couriers sigue un esquema de **compensación mínima garantizada** [13]: cada courier percibe el máximo entre su ganancia por entregas y un pago mínimo por hora:

$$\text{Pago}(c) = \max(p_1 \cdot n_c, p_2 \cdot (l_c - e_c)) \quad (2.1)$$

donde  $n_c$  es el número total de órdenes entregadas por el courier  $c$ ,  $p_1$  es el pago por orden y  $p_2$  el pago mínimo por hora. Este modelo alinea los incentivos del courier con los objetivos de la plataforma: cuando la demanda es suficiente, la ganancia variable domina; cuando la demanda es baja, la compensación mínima garantiza un ingreso base.

### 2. 1.4. Métricas de desempeño

Reyes et al. [13] definen 14 métricas primarias de desempeño para el MDRP, organizadas en torno a las perspectivas de los tres actores del sistema: clientes, restaurantes y repartidores. Las más relevantes para este trabajo son:

1. **Click-to-Door** (CtD): tiempo entre la colocación y la entrega de la orden,  $\text{CtD}_o = t_{\text{delivery},o} - a_o$ . Se define un objetivo  $\tau$  y un máximo absoluto  $\tau_{\text{max}}$ .
2. **Ready-to-Pickup** (RtP): tiempo de espera de la comida lista en el restaurante,  $\text{RtP}_o = t_{\text{pickup},o} - e_o$ .
3. **Ready-to-Door** (RtD): tiempo desde la disponibilidad hasta la entrega,  $\text{RtD}_o = t_{\text{delivery},o} - e_o$ .
4. **Utilización del courier**: fracción del turno que el repartidor dedica a conducción, recolección y entrega, en contraste con el tiempo de espera inactiva.
5. **Costo por orden**: compensación total dividida entre el número de órdenes entregadas.

El artículo original reporta estas métricas con estadísticas de resumen que incluyen media, desviación estándar, mínimo, percentil 10, mediana, percentil 90 y máximo. Sobre esa base, la Sección 3.7 detalla los 21 indicadores operativos implementados en esta tesis.

## 2. 1.5. Variantes del entorno operativo

Reyes et al. [13] identifican dos extensiones del modelo base que amplían las acciones disponibles para el despachador:

- **Preposicionamiento:** Además de asignar rutas de recolección-entrega, el despachador puede enviar a un courier hacia un restaurante anticipadamente (*prepositioning move*).
- **Actualización de asignaciones:** Cuando un courier llega a un restaurante, el conjunto de órdenes que recogerá *en ese mismo restaurante* puede modificarse (añadir o quitar órdenes) antes de la recolección efectiva. En el modelo de Reyes et al. [13], esta es la única actualización permitida: no se trata de una reoptimización arbitraria de toda la ruta, sino de un ajuste local del agrupamiento en el punto de recolección.

La Figura 2.1 resume visualmente ambas variantes.

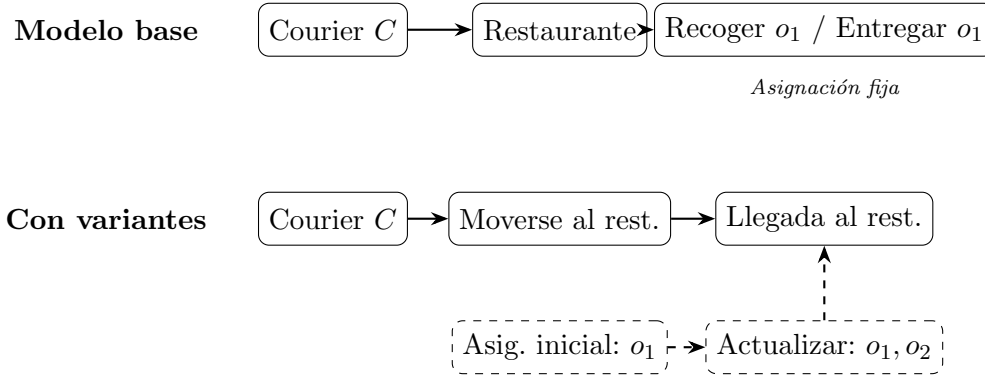


Figura 2.1: Comparación del modelo base frente al modelo con variantes (preposicionamiento y actualización local de asignaciones).

Ambas extensiones se implementan en el simulador mediante el mecanismo de **compromiso en dos etapas** descrito en la Sección 3.5: el compromiso parcial (*inbound*) actúa como preposicionamiento, y la reoptimización entre épocas permite actualizar agrupamientos antes de la recolección efectiva.

## 2. 1.6. Clasificación del MDRP en la literatura

El MDRP pertenece a la familia de los **problemas dinámicos de recolección y entrega** (dPDP), una subclase de los problemas dinámicos de ruteo de vehículos (dVRP) [10]. Siguiendo

la discusión de Reyes et al. [13], el MDRP puede ubicarse dentro de los *dynamic delivery problems*, y se distingue de otros dPDP por la concurrencia de varias propiedades: (1) instancias totalmente dinámicas (todas las órdenes llegan en línea), (2) alta urgencia con objetivos de servicio cortos ( $\tau \approx 40$  min), (3) baja flexibilidad operativa y (4) múltiples puntos de recolección (restaurantes). Estas propiedades hacen especialmente difícil el uso de métodos exactos en línea y justifican el empleo de heurísticas de horizonte rodante, como la desarrollada en la Sección 2.2.

## 2. 2. Heurística de horizonte rodante

El algoritmo de Reyes et al. [13] resuelve el MDRP mediante un esquema de **horizonte rodante** (*rolling horizon*) que, cada  $f$  minutos, ejecuta un ciclo de agrupamiento, asignación y compromiso sobre el subconjunto de órdenes cuyo *ready time* cae dentro del horizonte de asignación  $\Delta_u$ . Formalmente, al instante de optimización  $t$  el algoritmo opera sobre el conjunto

$$U_t = \{o \in \mathcal{O}_t : e_o \leq t + \Delta_u\},$$

donde  $\mathcal{O}_t$  es el conjunto de órdenes reveladas hasta  $t$  que aún no han sido finalizadas. Dado que planificar entregas para órdenes cuyo *ready time* es lejano es de escasa utilidad en un entorno altamente dinámico y urgente, restringir la atención a  $U_t$  reduce el espacio de decisiones sin sacrificar calidad de servicio.

### 2. 2.1. Tamaño objetivo de agrupamiento

Para equilibrar la utilización de los couriers con la frescura de las entregas, el algoritmo ajusta dinámicamente el número de órdenes que se agrupan en un mismo viaje. El **tamaño objetivo de agrupamiento** (*target bundle size*) al instante  $t$  se define como la razón entre las órdenes próximas a estar listas y los couriers disponibles:

$$Z_t = \frac{|\{o \in \mathcal{O}_t : e_o \leq t + \Delta_1\}|}{|\{c \in \mathcal{D}_t : e_c^{avail} \leq t + \Delta_2\}|}, \quad \Delta_1 > 0, \Delta_2 > 0, \quad (2.2)$$

donde  $\mathcal{D}_t$  es el conjunto de couriers en turno al instante  $t$ ,  $e_c^{avail}$  es el momento en que el courier  $c$  queda disponible para una nueva asignación, y  $\Delta_1, \Delta_2$  son horizontes de anticipación calibrados fuera de línea. Para que el algoritmo tenga carácter anticipatorio, Reyes et al. [13] recomiendan  $f < \Delta_u \leq \Delta_1$  y  $f < \Delta_u \leq \Delta_2$ . Cuando  $Z_t < 1$ , hay más couriers que órdenes

y el comportamiento converge hacia asignaciones individuales; cuando  $Z_t \gg 1$ , el sistema está saturado y se favorecen agrupamientos más grandes para maximizar la capacidad de entrega. Si no hay couriers disponibles antes de  $t + \Delta_2$ , se asigna un valor por defecto.

### 2. 2.2. Generación de agrupamientos

Una vez determinado  $Z_t$ , el algoritmo construye agrupamientos (*bundles*) por restaurante. Para cada restaurante  $r$ , se procesan las órdenes de  $U_{t,r} = U_t \cap \{o : r_o = r\}$  mediante un procedimiento de **inserción paralela** con mejora local [13]:

1. **Inicialización:** Se crean  $m_r = \max(k_t^r, \lceil |U_{t,r}|/Z_t \rceil)$  agrupamientos vacíos, donde  $k_t^r$  es el número de couriers disponibles en el restaurante  $r$ . Las órdenes se ordenan por  $e_o$  ascendente.<sup>1</sup>
2. **Inserción:** Cada orden se inserta en la posición del agrupamiento que minimiza el costo de inserción:

$$\text{costo}(s, o, p) = \sum_{(i,j) \in s} T(i, j) + \beta \sum_{i \in s} \text{ServiceDelay}(i), \quad (2.3)$$

donde  $T(i, j)$  es el tiempo de viaje entre ubicaciones consecutivas de la ruta y  $\beta > 0$  pondera la pérdida de frescura. Si un agrupamiento alcanza  $Z_t$  órdenes, una inserción adicional solo se permite si mejora la eficiencia (reduce el tiempo promedio por orden).

3. **Mejora local (*remove-reinsert*):** Cada orden se extrae de su agrupamiento actual y se reinserta en la posición globalmente óptima entre todos los agrupamientos, corrigiendo asignaciones sub-óptimas del paso anterior.

Cada agrupamiento queda asociado a una ruta abierta: restaurante  $\rightarrow$  entrega<sub>1</sub>  $\rightarrow$  entrega<sub>2</sub>  $\rightarrow \dots \rightarrow$  entrega<sub>k</sub>.

### 2. 2.3. Esquema de prioridad

Antes de resolver la asignación, los agrupamientos se clasifican en tres **grupos de prioridad** según su urgencia [13]:

---

<sup>1</sup>La implementación (Sección 3.5) utiliza  $m_r = \min(\lceil n/Z_t \rceil, |\mathcal{C}_{disp}|)$ , acotando el número de agrupamientos a los couriers disponibles para evitar agrupamientos sin courier posible.

- **Grupo I** (urgente): el objetivo de Click-to-Door  $\tau$  ya es inalcanzable para al menos una orden del agrupamiento.
- **Grupo II** (retrasado): el objetivo  $\tau$  aún es alcanzable, pero la recolección no puede realizarse en el *ready time* de las órdenes.
- **Grupo III** (normal): el agrupamiento puede recogerse y entregarse dentro de los objetivos de servicio.

La asignación se resuelve **secuencialmente** por grupo ( $I \rightarrow II \rightarrow III$ ), garantizando que los agrupamientos más urgentes obtengan los couriers más cercanos antes de que estos sean consumidos por órdenes menos críticas.

## 2. 2.4. Modelo de asignación lineal

En cada grupo de prioridad, la asignación de agrupamientos a couriers se formula como un **problema de asignación lineal** (*linear assignment problem*). Sea  $S$  el conjunto de agrupamientos del grupo actual y  $\mathcal{D}$  el subconjunto de couriers disponibles. Para cada par  $(s, d) \in S \times \mathcal{D}$  se define el peso:

$$w_{s,d} = \frac{N_s}{\max_{o \in s} \{\delta_{o,d}^s\} - e_d} - \theta \left( \pi_{s,d} - \max_{o \in s} \{e_o\} \right), \quad (2.4)$$

donde  $N_s$  es el número de órdenes en el agrupamiento  $s$ ,  $\delta_{o,d}^s$  es el tiempo de entrega de la orden  $o$  si  $s$  es asignado al courier  $d$ ,  $e_d$  es el momento en que  $d$  queda disponible,  $\pi_{s,d}$  es el tiempo de recolección del agrupamiento, y  $\theta > 0$  penaliza el retraso en la recolección respecto al *ready time*. El primer término captura la *eficiencia* (órdenes por unidad de tiempo); el segundo, la *frescura* (demora en el pickup).

El problema se formula como:

$$\max_x \quad \sum_{d \in \mathcal{D}} \sum_{s \in S} w_{s,d} x_{s,d} \quad (2.5)$$

$$\text{s.a.} \quad \sum_{s \in S} x_{s,d} \leq 1, \quad \forall d \in \mathcal{D} \quad (2.6)$$

$$\sum_{d \in \mathcal{D} \cup \{0\}} x_{s,d} = 1, \quad \forall s \in S \quad (2.7)$$

$$x_{s,d} \in \{0, 1\} \quad (2.8)$$

La restricción (2.6) garantiza que cada courier reciba a lo más un agrupamiento; la restricción (2.7) asegura que cada agrupamiento sea asignado a exactamente un courier (posiblemente artificial,  $d = 0$ , para absorber exceso de demanda). Al tratarse de un problema de asignación lineal, puede resolverse en  $O(n^3)$  mediante el **Algoritmo Húngaro** [?], lo que garantiza tiempos de ejecución despreciables incluso para instancias con cientos de couriers y agrupamientos.

Reyes et al. [13] exploran también formulaciones de mayor complejidad (partición de agrupamientos por *ready time* y pares de agrupamientos consecutivos), pero el modelo lineal constituye la formulación base del algoritmo y es la adoptada en la implementación de este trabajo (Sección 3.5).

### 2. 2.5. Estrategia de compromiso en dos etapas

Tras resolver la asignación, no todas las decisiones se comunican de forma inmediata a los couriers. La estrategia de **compromiso aditivo en dos etapas** (*two-stage additive commitment*) [13] descompone cada asignación tentativa  $(s, d)$  en:

1. **Compromiso final:** Si  $d$  puede alcanzar el restaurante  $r_s$  antes de  $t + f$  y todas las órdenes de  $s$  estarán listas antes de  $t + f$ , se instruye a  $d$  para recoger y entregar las órdenes de  $s$ .
2. **Compromiso parcial (*inbound*):** Si  $d$  no puede recoger  $s$  antes de  $t + f$ , pero completa su asignación actual antes de  $t + f$ , se le instruye únicamente a desplazarse hacia  $r_s$ . En la siguiente época,  $d$  se reasigna con la garantía de que su agrupamiento incluirá al menos las órdenes de  $s$  (propiedad aditiva).
3. **Ignorar:** Si  $d$  no puede iniciar una nueva asignación antes de  $t + f$ , la decisión se pospone.
4. **Excepción de espera prolongada:** Si alguna orden de  $s$  lleva lista más de  $x$  minutos, se fuerza un compromiso final independientemente de las condiciones anteriores.

El compromiso parcial actúa como *preposicionamiento*: el courier se acerca al restaurante mientras las órdenes terminan de prepararse, y el algoritmo retiene la flexibilidad de actualizar el agrupamiento cuando nueva información se revela en la siguiente época. Esta descomposición es consistente con las extensiones de preposicionamiento y actualización de asignaciones definidas en la Sección 2.1.5.



La Sección 3.5 describe la implementación concreta de este algoritmo, incluyendo los valores numéricos de los parámetros  $(f, \Delta_u, \Delta_1, \Delta_2, \beta, \theta, x)$  y las adaptaciones realizadas para la topología de La Paz.

## 2. 3. Enrutamiento sobre grafos viales

Las heurísticas descritas en la Sección 2.2 requieren evaluar tiempos de viaje entre pares de ubicaciones en cada época de optimización. Reyes et al. [13] emplean distancias euclidianas multiplicadas por un factor de conversión  $\gamma$ , lo que produce un grafo completo con tiempos de viaje simétricos. No obstante, Boyacı et al. [12] demuestran que la aproximación euclidiana introduce errores sistemáticos del 17–40% respecto a las distancias reales sobre la red vial, especialmente en topologías irregulares. Para eliminar esta fuente de error, el presente trabajo sustituye el modelo euclidiano por consultas de enrutamiento sobre el grafo vial real, empleando la cadena `OpenStreetMap`  $\rightarrow$  `OSRM`.

### 2. 3.1. OpenStreetMap como modelo de datos geográficos

**OpenStreetMap** (OSM) es un proyecto colaborativo de cartografía abierta que modela la infraestructura vial mediante tres primitivas [17]:

- **Nodos** (*nodes*): puntos georreferenciados (*lat, lon*) que representan intersecciones, semáforos o puntos de interés.
- **Vías** (*ways*): secuencias ordenadas de nodos que representan segmentos de calle. Cada vía porta atributos (*tags*) como tipo de vía (**highway**), límite de velocidad (**maxspeed**), sentido de circulación (**oneway**) y superficie (**surface**).
- **Relaciones** (*relations*): agrupaciones de nodos y vías que representan estructuras complejas (restricciones de giro, rutas de transporte público, etc.).

El grafo vial resultante  $G = (V, E, c)$  tiene vértices  $V$  correspondientes a los nodos y aristas  $E$  derivadas de las vías, con función de costo  $c(e)$  definida por la distancia geográfica del segmento y la velocidad permitida. Para la zona urbana de La Paz, B.C.S., el extracto de Geofabrik contiene aproximadamente  $10^4$  nodos y  $10^4$  aristas relevantes para el perfil vehicular.

### 2. 3.2. Contraction Hierarchies

Dado un grafo vial  $G = (V, E, c)$ , el cálculo de caminos mínimos mediante el algoritmo de Dijkstra tiene complejidad  $O(|E| + |V| \log |V|)$ , lo que resulta prohibitivo para consultas frecuentes en grafos con millones de nodos. Geisberger et al. [18] proponen **Contraction Hierarchies** (CH), una técnica de preprocesamiento que permite resolver consultas de camino mínimo en milisegundos.

El algoritmo se divide en dos fases:

**Preprocesamiento.** Los nodos del grafo se ordenan por un criterio de *importancia* (e.g., diferencia de aristas creadas menos aristas eliminadas) y se contraen secuencialmente. Contraer un nodo  $v$  consiste en:

1. Eliminar  $v$  y todas sus aristas incidentes del grafo.
2. Para cada par de vecinos  $(u, w)$  de  $v$ , si el camino mínimo  $u \rightarrow v \rightarrow w$  es el único camino óptimo entre  $u$  y  $w$  en el grafo residual, insertar una **arista atajo** (*shortcut*)  $(u, w)$  con peso  $c(u, v) + c(v, w)$ .

El resultado es un grafo aumentado  $G^+ = (V, E \cup E_{sc})$  donde  $E_{sc}$  es el conjunto de atajos. Cada nodo recibe un nivel jerárquico igual a su orden de contracción.

**Consulta.** Para encontrar el camino mínimo de  $s$  a  $t$ , se ejecuta una **búsqueda bidireccional** en  $G^+$ : desde  $s$  hacia arriba en la jerarquía (solo aristas hacia nodos de mayor nivel) y desde  $t$  hacia arriba. El camino óptimo pasa por el nodo de encuentro que minimiza la suma de distancias parciales. La complejidad empírica de la consulta es  $O(\sqrt{|V|} \cdot \log |V|)$ , lo que en la práctica permite resolver consultas en redes continentales en menos de 1 ms [18].

### 2. 3.3. OSRM: motor de enrutamiento

**OSRM** (*Open Source Routing Machine*) es un motor de enrutamiento de código abierto que implementa CH sobre datos de OpenStreetMap. Su cadena de procesamiento consta de tres etapas:

1. **osrm-extract**: parsea el archivo `.pbf` de OSM y genera una representación intermedia del grafo, aplicando un perfil de velocidades por tipo de vía (e.g., `car.lua`).

2. **osrm-contract**: construye la jerarquía CH sobre el grafo extraído, generando archivos de índice que codifican los atajos y la ordenación jerárquica.
3. **osrm-routed**: servidor HTTP que responde consultas de ruta punto a punto (`/route/v1/driving/{c` con tiempos de respuesta típicos  $< 5$  ms para grafos regionales.

El desacople entre preprocesamiento (offline, una sola vez) y consulta (online, por época) es esencial para el simulador: la fase de contracción se ejecuta una vez sobre el extracto cartográfico, y cada época  $t$  del algoritmo rolling horizon emite  $O(|\mathcal{C}| \cdot |S|)$  consultas al servidor, que se resuelven individualmente en tiempo constante amortizado.

La sustitución de la distancia euclidiana por tiempos OSRM tiene tres consecuencias directas sobre las heurísticas de la Sección 2.2:

- Los tiempos de viaje dejan de ser simétricos:  $T(a, b) \neq T(b, a)$  por sentidos de circulación.
- La desigualdad triangular se cumple de forma natural sobre la red vial, sin necesidad de correcciones *ad hoc*.
- El factor de conversión  $\gamma$  de Reyes et al. [13] se elimina: los tiempos provienen directamente de las velocidades y topología reales.

La Sección 3.1 detalla la configuración concreta del contenedor Docker, el perfil de velocidades utilizado y la integración con el módulo `getrouteOSMR.py`.

## 2. 4. Generación estocástica de demanda y métricas

La evaluación de políticas de despacho requiere instancias de prueba cuyas llegadas reproduzcan la variabilidad temporal observada en operaciones reales. Esta sección presenta el marco probabilístico que sustenta la generación de demanda y formaliza las familias de indicadores de desempeño utilizadas para comparar las políticas.

### 2. 4.1. Proceso de Poisson no homogéneo

Un **proceso de Poisson no homogéneo** (PPNH) es un proceso de conteo  $\{N(t), t \geq 0\}$  en el que el número de eventos en un intervalo  $(t_1, t_2]$  sigue una distribución de Poisson con parámetro igual a la integral de una función de intensidad  $\lambda(\cdot)$  [? ]:

$$N(t_2) - N(t_1) \sim \text{Poisson}\left(\int_{t_1}^{t_2} \lambda(u) du\right), \quad \lambda(u) > 0 \quad (2.9)$$

con incrementos independientes en intervalos disjuntos. A diferencia del proceso de Poisson homogéneo, la tasa  $\lambda(t)$  varía con el tiempo, lo que permite capturar los picos y valles de demanda característicos de los servicios de entrega de comida.

**Propiedad de discretización.** Cuando la intensidad se integra sobre intervalos unitarios  $[t, t+1)$ , el conteo en cada intervalo hereda la distribución de Poisson:

$$N_t \sim \text{Poisson}\left(\int_t^{t+1} \lambda(u) du\right) \approx \text{Poisson}(\lambda(t)),$$

aproximación válida si  $\lambda$  varía lentamente dentro del intervalo. Este resultado justifica el muestreo minuto a minuto empleado en la Sección 3.2.

## 2. 4.2. Función de intensidad bimodal

Los patrones de demanda en servicios de entrega de comida exhiben dos picos diarios correspondientes a los horarios de almuerzo y cena [16]. Una representación parsimoniosa de esta estructura bimodal es la superposición de una intensidad base constante y dos componentes gaussianas:

$$\lambda(t) = \lambda_0 + A_m \exp\left(-\frac{(t - \mu_m)^2}{2\sigma^2}\right) + A_e \exp\left(-\frac{(t - \mu_e)^2}{2\sigma^2}\right) \quad (2.10)$$

donde  $\lambda_0$  es la intensidad base,  $\mu_m$  y  $\mu_e$  son los centros temporales de los picos matutino y vespertino,  $A_m$  y  $A_e$  las amplitudes respectivas, y  $\sigma$  la dispersión temporal compartida. Los valores concretos de estos parámetros se calibran a partir de los perfiles de demanda del dataset LaDe [16]; la Sección 3.2 detalla la calibración específica y el procedimiento de muestreo implementados para La Paz.

La elección del PPNH con perfil gaussiano bimodal se justifica por tres razones:

1. **Parsimonia:** seis parámetros ( $\lambda_0, A_m, A_e, \sigma, \mu_m, \mu_e$ ) capturan la esencia del patrón diario sin sobreajuste.
2. **Flexibilidad:** escalar  $\lambda_0$  permite variar el volumen total de órdenes manteniendo la forma temporal.

3. **Tratabilidad:** la propiedad de discretización permite generar instancias mediante un simple muestreo de Poisson por minuto, sin necesidad de algoritmos de adelgazamiento (*thinning*).

## 2. 4.3. Métricas de desempeño: marco de evaluación

La Sección 2.1 introdujo las métricas primarias del MDRP definidas por Reyes et al. [13]: Click-to-Door, Ready-to-Pickup, Ready-to-Door, utilización del courier y costo por orden. Para una evaluación cuantitativa rigurosa, estas métricas individuales (por orden o por courier) se agregan en estadísticos de resumen y se organizan en cinco **familias** que cubren las perspectivas de los actores del sistema.

**Estadísticos de agregación.** Para cada métrica a nivel de orden (e.g.,  $CtD_o$ ), se calculan:

- **Media**  $\bar{m} = \frac{1}{n} \sum_{o=1}^n m_o$ , que resume el comportamiento central.
- **Percentiles**  $P_{50}$ ,  $P_{90}$ ,  $P_{95}$ : el percentil 90 es particularmente relevante porque captura la experiencia de los clientes en la cola de la distribución, donde se concentran las entregas con mayor demora.

**Familias de indicadores.** Las métricas se clasifican en cinco dimensiones:

1. **Calidad de servicio.** CtD (media,  $P_{50}$ ,  $P_{90}$ ,  $P_{95}$ ), RtP (media,  $P_{90}$ ), RtD (media,  $P_{90}$ ), tasa de cumplimiento SLA:

$$SLA = \frac{|\{o : CtD_o \leq \tau\}|}{|O_{del}|} \times 100\% \quad (2.11)$$

y exceso promedio respecto al objetivo  $\bar{\Delta} = \frac{1}{n} \sum_o \max(0, CtD_o - \tau)$ .

2. **Cobertura.** Fracción de órdenes no entregadas al cierre de la simulación.
3. **Eficiencia operativa.** Pedidos por repartidor por hora, agrupamientos por hora, utilización (% del turno conduciendo), distancia total y distancia por pedido.
4. **Eficiencia de agrupamiento.** Tamaño promedio de agrupamiento y proporción de órdenes en agrupamientos múltiples, indicadores que reflejan el grado de consolidación logrado por la política de *bundling*.

5. **Costo.** Compensación total, costo por pedido y fracción de repartidores con compensación mínima garantizada, derivadas del modelo de compensación dual de la Ecuación 3.5.

El presente trabajo implementa 21 indicadores distribuidos en estas cinco familias; la Sección 3.7 detalla la definición operativa de cada uno y la Tabla 3.4 los enumera con su formulación abreviada.

# Capítulo 3

## Desarrollo

Este capítulo describe la implementación computacional del entorno de simulación para el MDRP en La Paz, B.C.S. Se documenta la arquitectura del sistema, la generación de instancias sintéticas, el motor de simulación por épocas, las dos políticas de despacho implementadas (FCFS y Rolling Horizon), el diseño experimental y las métricas de evaluación. Cada sección establece la correspondencia entre los conceptos teóricos del Capítulo 2 y su realización en código.

### 3. 1. Arquitectura del sistema

El sistema se organiza en tres capas funcionales desacopladas: (1) una capa de **enrutamiento** basada en OSRM contenerizado, (2) una capa de **simulación** implementada en Python, y (3) una capa de **análisis** que recolecta y procesa los indicadores de desempeño. La Figura 3.1 ilustra la interacción entre estos componentes.

#### 3. 1.1. Capa de enrutamiento: OSRM contenerizado

La capa de enrutamiento proporciona tiempos y distancias de viaje realistas sobre la red vial de La Paz. Se implementa mediante OSRM (*Open Source Routing Machine*) v5.26.0, desplegado como un contenedor Docker orquestado por `docker-compose`.

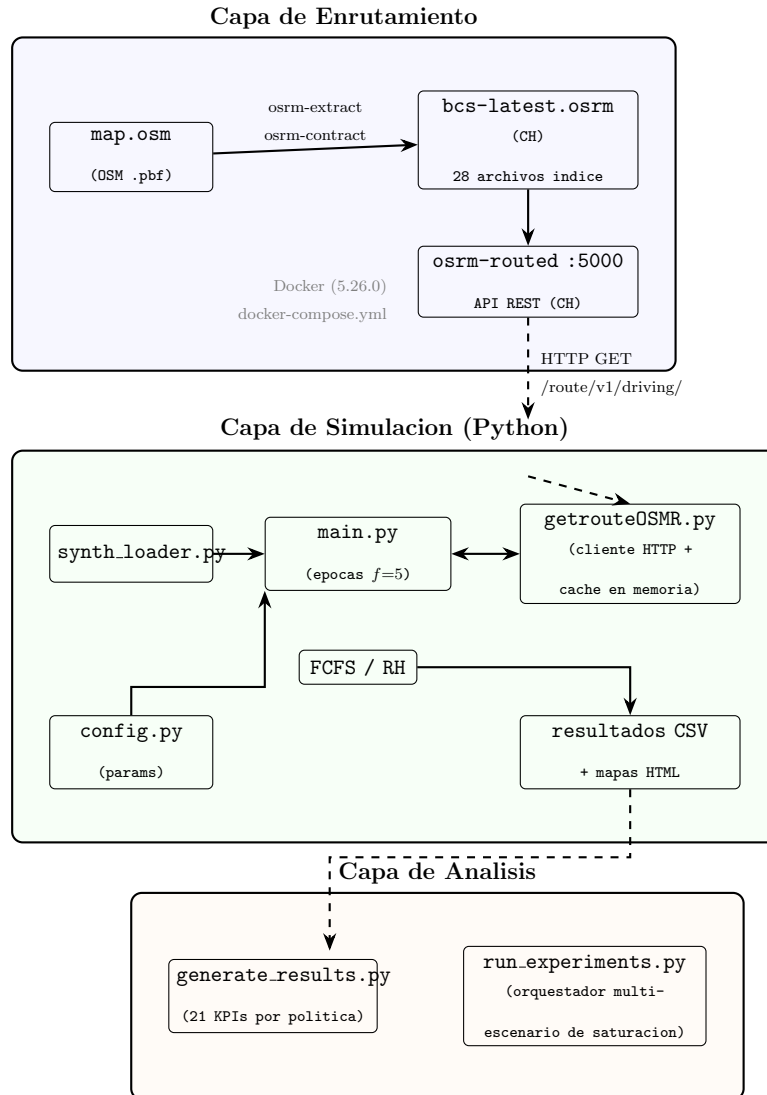


Figura 3.1: Arquitectura de tres capas del entorno de simulacion MDRP-BCS.



## Preprocesamiento de la red vial

El proceso de construcción de la red vial sigue la cadena de preprocesamiento estándar de OSRM. A partir del extracto cartográfico de OpenStreetMap para Baja California Sur (`bcs-latest.osm.pbf`), se ejecutan dos etapas secuenciales:

1. **Extracción** (`osrm-extract`): Parsea el archivo `.pbf` y genera una representación intermedia del grafo vial, aplicando el perfil de velocidades del vehículo (`car.lua`). Este perfil define las velocidades máximas permitidas por tipo de vía (autopista, avenida, calle residencial), así como las restricciones de giro y sentido.
2. **Contracción** (`osrm-contract`): Construye la jerarquía de Contraction Hierarchies (CH) [18] sobre el grafo extraído. El algoritmo asigna un *orden de importancia* a cada nodo y genera atajos (*shortcuts*) que permiten saltar nodos intermedios durante las consultas. El resultado son 28 archivos de índice (`bcs-latest.osrm.*`) que codifican la jerarquía completa.

Estos archivos preprocesados se incluyen en el repositorio bajo el directorio `osrm_data/`, evitando que los usuarios deban repetir el preprocesamiento (que requiere aproximadamente 10–15 minutos y el archivo `.pbf` original de ~30 MB).

Es importante señalar que un archivo `.pbf` constituye una versión congelada de la cartografía en un momento dado. Aunque OpenStreetMap se actualiza de forma continua incorporando nuevas calles, colonias y modificaciones viales mediante contribuciones colaborativas, los archivos preprocesados no reflejan estos cambios automáticamente. Si se desea incorporar actualizaciones cartográficas (por ejemplo, la apertura de un nuevo tramo vial en La Paz), es necesario descargar un `.pbf` más reciente y repetir la cadena completa de preprocesamiento (`osrm-extract` → `osrm-contract` → reinicio de `osrm-routed`). Para garantizar la reproducibilidad de los experimentos, el presente trabajo congela la versión del mapa utilizada; el extracto empleado corresponde a la descarga de Geofabrik para Baja California Sur con fecha de corte diciembre de 2025. Cualquier replicación futura debe utilizar este mismo archivo o documentar explícitamente la fecha del nuevo extracto.

## Despliegue con Docker Compose

El servicio de enrutamiento se despliega de forma declarativa mediante un archivo `docker-compose.yml` que encapsula toda la configuración necesaria:

Listing 3.1: Definición del servicio OSRM en `docker-compose.yml`.

```
services:
  osrm:
    image: osrm/osrm-backend:5.26.0
    container_name: mdrp-osrm
    ports:
      - "5000:5000"
    volumes:
      - ./osrm_data:/data:ro
    command: osrm-routed --algorithm ch
      /data/bcs-latest.osrm
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f",
        "http://localhost:5000/route/v1/driving/
        -110.31,24.14;-110.30,24.15"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Esta configuración garantiza tres propiedades esenciales para la reproducibilidad:

- **Aislamiento del entorno:** La imagen Docker `osrm/osrm-backend:5.26.0` fija la versión exacta del motor de enrutamiento, eliminando discrepancias por versiones de librerías del sistema operativo anfitrión.
- **Inmutabilidad de datos:** El volumen se monta en modo de solo lectura (`:ro`), garantizando que los archivos de la red vial preprocesada no sean modificados accidentalmente durante la ejecución.

- **Verificación de salud:** El `healthcheck` ejecuta una consulta de enrutamiento real cada 30 segundos, asegurando que el servicio esté operativo antes de que el simulador inicie las consultas. La coordenada de prueba  $(-110.31, 24.14)$  corresponde a un punto dentro de la zona urbana de La Paz.

El despliegue se reduce a un único comando: `docker-compose up -d`. El contenedor expone el puerto 5000 en `localhost`, donde la API REST de OSRM queda disponible para recibir consultas del simulador Python.

### API REST y protocolo de consulta

El simulador consume la API de OSRM mediante solicitudes HTTP GET al *endpoint* `/route/v1/driving/`. El módulo `getrouteOSMR.py` implementa el cliente HTTP con las siguientes características:

- **Formato de consulta:** Las coordenadas se especifican como pares longitud/latitud separados por punto y coma. Por ejemplo, una ruta desde un restaurante hacia dos puntos de entrega se codifica como: `/route/v1/driving/lon1,lat1;lon2,lat2;lon3,lat3`
- **Respuesta JSON:** OSRM retorna la distancia total (metros), duración total (segundos), la geometría de la ruta codificada en formato *polyline*, y los detalles por tramo (*legs*) incluyendo las maniobras de cada paso.
- **Fallback euclidiano:** Si la consulta OSRM falla (por desconexión o error de red), el sistema calcula una estimación basada en la distancia de Haversine a una velocidad constante configurable (por defecto 320 m/min  $\approx$  19.2 km/h). Este mecanismo, controlado por la variable de entorno `USE_EUCLIDEAN_ON_FAILURE`, garantiza que la simulación no se interrumpa, aunque los resultados con fallback euclidiano se señalan en la salida para que el investigador los detecte.

### 3. 1.2. Capa de simulación: estructura de módulos Python

La lógica de simulación se implementa en Python 3.12 y se estructura en seis módulos bajo el directorio `src/`, cada uno con una responsabilidad bien definida. La Tabla 3.1 resume la función de cada módulo.

Tabla 3.1: Módulos del simulador y su responsabilidad.

| Módulo                              | Responsabilidad                                     |
|-------------------------------------|-----------------------------------------------------|
| <code>config.py</code>              | Parámetros globales de la simulación                |
| <code>synth_loader.py</code>        | Carga de instancias y dimensionamiento de flota     |
| <code>main.py</code>                | Motor de simulación por épocas y modelos de datos   |
| <code>getrouteOSMR.py</code>        | Cliente HTTP para OSRM con caché y fallback         |
| <code>bundling.py</code>            | Generación de agrupamientos por restaurante         |
| <code>asignaciontentativa.py</code> | Asignación courier-agrupamiento (Algoritmo Húngaro) |

La Tabla 3.2 presenta los parámetros de configuración centralizados en `config.py`, que controlan el comportamiento de la simulación y permiten la experimentación paramétrica sin modificar el código fuente.

### 3. 1.3. Capa de análisis: pipeline de resultados

La tercera capa del sistema procesa las salidas crudas de la simulación y genera las métricas comparativas. Se compone de dos scripts complementarios:

- `generate_results.py`: Orquesta el pipeline completo para un escenario individual. Ejecuta secuencialmente la generación de datos sintéticos, la simulación FCFS, la simulación Rolling Horizon, y el cálculo de 21 KPIs comparativos. Los resultados se consolidan en `results/kpi_comparison.csv`.
- `run_experiments.py`: Amplía el pipeline anterior para ejecutar múltiples escenarios de saturación de forma automatizada. Define tres niveles de relación órdenes/courier (18:1, 26:1 y 35:1), ejecuta ambas políticas en cada escenario, y genera una tabla consolidada (`experiment_comparison.csv`) que reúne los KPIs de todos los escenarios. Adicionalmente, soporta la ejecución con múltiples semillas aleatorias (`--seeds`) para análisis y la variación de parámetros. (`--bundle-sizes`).

La separación en tres capas independientes permite modificar cualquier componente sin afectar a los demás. Por ejemplo, sustituir OSRM por otro motor de enrutamiento solo requiere modificar `getrouteOSMR.py`, sin alterar la lógica de simulación ni el cálculo de métricas. De

Tabla 3.2: Parámetros de configuración del simulador (`config.py`).

| Parámetro                     | Símbolo      | Valor  | Descripción                     |
|-------------------------------|--------------|--------|---------------------------------|
| Frecuencia de optimización    | $f$          | 5 min  | Intervalo entre épocas          |
| Horizonte de asignación       | $\Delta_u$   | 20 min | Horizonte para órdenes          |
| Objetivo Click-to-Door        | $\tau$       | 40 min | Meta de calidad de servicio     |
| Máximo Click-to-Door          | $\tau_{max}$ | 90 min | Límite absoluto de servicio     |
| Tiempo de servicio            | $s$          | 4 min  | Pickup + drop-off por parada    |
| Tamaño máximo de agrupamiento | —            | 4      | Límite de órdenes por ruta      |
| Penalización de frescura      | $\beta$      | 0.1    | Peso del retraso en asignación  |
| Penalización de pickup        | $\theta$     | 0.1    | Peso del retraso en recolección |
| Horizonte de ready time       | $\Delta_1$   | 20 min | Ventana de anticipación         |
| Horizonte de disponibilidad   | $\Delta_2$   | 20 min | Ventana para couriers           |
| Forzamiento de compromiso     | $x$          | 10 min | Commit si orden espera $> x$    |
| Pago por orden                | $p_1$        | \$10   | Compensación variable           |
| Pago mínimo por hora          | $p_2$        | \$15/h | Compensación garantizada        |

manera análoga, agregar nuevos escenarios experimentales no requiere cambios en el simulador ni en la infraestructura de enrutamiento.

## 3. 2. Generación de instancias sintéticas

Dado que no existen registros históricos públicos de operaciones de entrega de comida en La Paz, B.C.S., el sistema genera instancias sintéticas calibradas que replican los patrones temporales y geoespaciales documentados en la literatura. El proceso se divide en dos etapas: (1) la generación de órdenes (`make_synth_orders.py`) y (2) la carga y dimensionamiento de flota (`synth_loader.py`).

### 3. 2.1. Generación temporal de órdenes: proceso Poisson no homogéneo

### 3. 2.2. Tiempo de preparación

Cada orden recibe un **tiempo de preparación**  $T_{prep}$  que determina cuándo la comida estará lista para ser recogida por el courier. Los valores  $\mu_{prep} = 12$  y  $\sigma_{prep} = 3$  se seleccionaron a partir de los rangos de preparación observados en instancias publicadas del MDRP [13] y en estudios empíricos de operaciones de última milla en entrega de comida [? ]. El truncamiento inferior en 5 minutos evita tiempos de preparación irrealistamente cortos. El *ready time* de cada orden se calcula como:

$$t_{ready} = t_{created} + T_{prep} \quad (3.1)$$

donde  $t_{created} = a_o$  es el instante de colocación de la orden (`placement_time`). Este ready time constituye una **restricción dura** en el modelo de simulación: ningún courier puede recoger la orden antes de  $t_{ready}$ , conforme a la definición del MDRP en Reyes et al. [13].

### 3. 2.3. Distribución geoespacial

Las ubicaciones de restaurantes y clientes se generan mediante **muestreo uniforme** dentro de un polígono que delimita la zona urbana de La Paz. Este polígono se define por cuatro vértices que encierran el área de mayor densidad poblacional y comercial:

$$\mathcal{P} = \text{Polígono} \left( \begin{bmatrix} 24.124 \\ -110.312 \end{bmatrix}, \begin{bmatrix} 24.133 \\ -110.336 \end{bmatrix}, \begin{bmatrix} 24.159 \\ -110.310 \end{bmatrix}, \begin{bmatrix} 24.147 \\ -110.293 \end{bmatrix} \right) \quad (3.2)$$

El muestreo utiliza un método de **rechazo** (*rejection sampling*): se generan puntos uniformemente en el rectángulo delimitador del polígono y se descartan aquellos que caen fuera de  $\mathcal{P}$  (por ejemplo, sobre la bahía de La Paz). Esto garantiza que todas las ubicaciones generadas correspondan a puntos terrestres dentro de la mancha urbana.

## Asignación restaurante-cliente

### Fuente de datos de restaurantes

El número de restaurantes se determina a partir del archivo `la_paz_restaurants.geojson`, que contiene 25 establecimientos de comida en La Paz extraídos de OpenStreetMap [17]. Para cada ejecución, se generan 25 ubicaciones aleatorias dentro del polígono urbano  $\mathcal{P}$  mediante el mismo procedimiento de muestreo por rechazo descrito anteriormente; las coordenadas originales de OSM no se conservan. De este modo se preserva la cardinalidad real de la oferta gastronómica de La Paz mientras se garantiza la consistencia geográfica con la zona de operación delimitada.

### 3. 2.4. Dimensionamiento automático de flota

El módulo `synth_loader.py` transforma el CSV de órdenes generado en objetos de simulación (`Order`, `Courier`, `Restaurant`) y calcula automáticamente el tamaño de la flota de couriers. La fórmula de dimensionamiento es:

$$n_{couriers} = \max \left( n_{min}, \left\lceil \frac{|\mathcal{O}|}{r} \right\rceil \right) \quad (3.3)$$

donde  $|\mathcal{O}|$  es el número total de órdenes,  $r$  es el ratio objetivo de órdenes por courier (`orders_per_courier`, por defecto  $r = 18$ ), y  $n_{min}$  es un piso mínimo de flota (`min_couriers`, por defecto  $n_{min} = 15$ ; en el diseño experimental de la Sección 3.6 se emplea  $n_{min} = 10$ ). Este mecanismo permite variar el nivel de **saturación** de la flota: valores altos de  $r$  producen flotas más pequeñas y mayor carga por courier, mientras que valores bajos generan flotas holgadas con baja utilización.

## Configuración de turnos

Todos los couriers comparten un turno uniforme que se calcula a partir de los datos:

- **Inicio del turno:** 15 minutos antes de la primera orden colocada, permitiendo que los couriers estén disponibles cuando arriben las primeras solicitudes.
- **Fin del turno:** 1 hora después del último *ready time*, garantizando tiempo suficiente para completar las entregas finales.
- **Ubicación inicial:** Todos los couriers parten del centroide geográfico de los restaurantes, representando un punto de espera central antes de la primera asignación.

## Reproducibilidad

El generador acepta una **semilla aleatoria** (`--seed`, por defecto 2025) que inicializa el generador de números pseudoaleatorios de NumPy (`numpy.random.default_rng`). Esto garantiza que, dada la misma semilla y el mismo objetivo de órdenes, se produzca una instancia idéntica en cualquier máquina. El script `run_experiments.py` aprovecha esta propiedad para ejecutar múltiples réplicas con semillas distintas.

## 3. 3. Motor de simulación

El motor de simulación (`main.py`) implementa un ciclo discreto dirigido por épocas que avanza el reloj virtual en incrementos de  $f = 5$  min. En cada época, el motor ejecuta cuatro fases secuenciales: **activación**, **revelación**, **asignación** y **actualización**.

### 3. 3.1. Ciclo de simulación por épocas

Sea  $t$  el tiempo de simulación actual. El ciclo se inicia en  $t_0$  (15 min antes de la primera orden) y termina en  $t_{end}$  (1 h después del último *ready time*). En cada iteración se ejecuta:

1. **Activación de couriers:** Se recorre la lista completa de couriers y se incorporan al conjunto de couriers activos  $\mathcal{C}_{act}$  aquellos cuyo *on-time*  $e_c \leq t$ :

$$\mathcal{C}_{act}(t) = \{c \in \mathcal{C} \mid e_c \leq t\} \quad (3.4)$$



2. **Revelación de órdenes:** Las órdenes cuyo *placement time*  $a_o \leq t$  se extraen de la cola ordenada (*deque*) y pasan al estado **ready**, quedando disponibles para asignación. Cada orden se registra en la lista de órdenes pendientes de su restaurante.
3. **Asignación:** Cuando  $t$  es múltiplo de  $f$ , se activa la lógica de despacho (FCFS o Rolling Horizon, según la variable de entorno `FCFS_POLICY`). La selección de política se realiza en tiempo de ejecución, permitiendo ejecutar ambas sobre la misma instancia sin modificar código.
4. **Actualización de rutas:** Se verifica si algún courier ha completado su ruta activa ( $t \geq t_{completion}$ ). Para rutas finalizadas, se registran los tiempos de pickup y delivery de cada orden, se actualiza la ubicación del courier al último waypoint, se acumulan distancia y tiempo de conducción, y se libera al courier para futuras asignaciones.

Finalmente, el reloj avanza  $t \leftarrow t + f$  y el ciclo se repite. Al terminar la simulación, se calcula la compensación final de cada courier.

### 3. 3.2. Modelos de datos: órdenes, couriers y restaurantes

El motor define tres clases que encapsulan el estado de las entidades durante la simulación:

- **Order:** Almacena `placement_time`, `ready_time`, `dropoff_loc`, el restaurante de origen, y campos mutables: `status` (`pending`  $\rightarrow$  `ready`  $\rightarrow$  `assigned`  $\rightarrow$  `delivered`), `pickup_time`, `delivery_time`, `courier_id` y `bundle.size`. Expone métodos para calcular el Click-to-Door ( $CtD = t_{delivery} - t_{placement}$ ) y el Ready-to-Pickup ( $RtP = t_{pickup} - t_{ready}$ ).
- **Courier:** Almacena `on_time`, `off_time`, ubicación actual, ruta activa (`current_route`) e historial de rutas completadas. Acumula métricas operativas: órdenes entregadas, distancia total (km), tiempo de conducción (min) y agrupamientos recogidos. El método `final_compensation()` implementa el modelo de pago dual:

$$\text{Compensación} = \max(p_1 \cdot n_{orders}, p_2 \cdot h_{shift}) \quad (3.5)$$

donde  $p_1 = \$10/\text{orden}$ ,  $p_2 = \$15/\text{h}$ , y  $h_{shift}$  es la duración del turno en horas.

- **Restaurant:** Almacena identificador, ubicación ( $lat, lon$ ) y la lista de órdenes pendientes. Sirve como punto de agrupación para el módulo de *bundling*.

### 3. 3.3. Máquina de estados de una orden

Cada orden transita por cuatro estados durante su ciclo de vida:

1. **pending**: La orden ha sido colocada pero su *placement time* aún no ha sido alcanzado por el reloj de simulación. La orden permanece en la cola ordenada.
2. **ready**: El reloj ha alcanzado o superado  $a_o$  (*placement time*). La orden se ha revelado al sistema y se registra en la lista de órdenes pendientes de su restaurante. Nótese que este estado indica visibilidad para el despachador, no que la comida esté físicamente preparada; la restricción de preparación se verifica por separado mediante el atributo **ready\_time** ( $t_{ready}$ ) en la fase de compromiso (Sección 3.5.4).
3. **assigned**: La orden ha sido asignada a un courier y forma parte de su ruta activa. En la política FCFS, esta transición ocurre de forma explícita al seleccionar al courier más cercano. En la política Rolling Horizon, la orden se incorpora al agrupamiento del courier sin cambio explícito de estado.
4. **delivered**: Un courier ha completado la entrega. Se registran  $t_{pickup}$  y  $t_{delivery}$ , y la orden se mueve a la lista de órdenes entregadas para el cálculo posterior de KPIs.

Las transiciones **pending**  $\rightarrow$  **ready** ocurren en la fase de revelación de cada época. La transición **ready**  $\rightarrow$  **assigned** ocurre en la fase de asignación, y la transición **assigned**  $\rightarrow$  **delivered** ocurre en la fase de actualización, cuando el courier completa su ruta.

### 3. 3.4. Integración con OSRM

El motor se comunica con el servicio de enrutamiento a través de `getrouteOSMR.py`, que actúa como capa de abstracción entre la simulación y la API REST. Cada vez que un módulo de asignación (FCFS o Rolling Horizon) necesita evaluar una ruta candidata, invoca `get_route_details(origin, waypoints)`, que:

1. Verifica si la consulta existe en la caché en memoria (`_osrm_cache`), indexada por la tupla `(origin, waypoints)`.
2. Si hay *cache hit*, retorna el resultado almacenado sin consulta HTTP.

3. Si hay *cache miss*, construye la URL con coordenadas en formato `lon,lat` y realiza la solicitud GET al contenedor OSRM.
4. Si la solicitud falla, aplica el fallback euclidiano (Haversine a 320 m/min).
5. Almacena el resultado en caché para consultas futuras.

La respuesta de OSRM se estructura como un diccionario con campos `distance` (metros), `duration` (segundos), `geometry` (polyline codificado) y `legs` (tramos individuales). Estos valores alimentan directamente el cálculo de tiempos de pickup, delivery y los KPIs de distancia recorrida.

### 3. 4. Política FCFS (baseline)

La política **First-Come, First-Served** (FCFS) sirve como línea base contra la cual se evalúa el desempeño de la política Rolling Horizon. Implementa una lógica de despacho reactiva sin agrupamiento ni optimización global: cada orden se asigna individualmente al courier disponible más cercano en el momento de la decisión.

Un aspecto central del diseño experimental es que FCFS opera **sin acceso al motor de enrutamiento vial** (OSRM). Tanto la selección del courier como la estimación de tiempos de recorrido se realizan exclusivamente mediante la **distancia geodésica de Haversine** a una velocidad constante de 320 m/min ( $\approx 19.2$  km/h). Esta decisión de diseño (`USE_EUCLIDEAN=1`) se fundamenta en dos observaciones:

1. **Consistencia con la literatura.** Las instancias públicas del MDRP propuestas por Reyes et al. [13] calculan los tiempos de viaje como el producto de la distancia euclidiana por un multiplicador  $\gamma$  (velocidad inversa), sin recurrir a grafos viales. La política FCFS de este trabajo replica esa misma filosofía de estimación simplificada.
2. **Representatividad operativa.** La mayoría de las plataformas de entrega de comida emergentes —especialmente las que operan en ciudades pequeñas y medianas— no integran un motor de enrutamiento sobre la red vial y estiman distancias y tiempos a partir de la línea recta entre coordenadas geográficas. FCFS modela esta práctica como cota inferior realista.

De este modo, la comparación FCFS vs. Rolling Horizon captura conjuntamente dos fuentes de mejora: (1) el uso de tiempos de viaje realistas basados en la red vial (OSRM) y (2) la optimización combinatoria de agrupamiento y asignación.

### 3. 4.1. Lógica de asignación

En cada época de asignación ( $t$  múltiplo de  $f$ ), la política FCFS ejecuta el siguiente procedimiento para cada orden  $o$  con estado **ready**:

1. **Filtrar couriers disponibles:** Se construye el conjunto de couriers sin ruta activa cuyo turno no haya expirado:

$$\mathcal{C}_{disp}(t) = \{c \in \mathcal{C}_{act}(t) \mid c.current\_route = \emptyset \wedge l_c > t\} \quad (3.6)$$

donde  $l_c$  es el fin del turno del courier (**off\_time**).

2. **Selección por proximidad:** Para cada courier disponible se calcula la distancia euclidiana (coordenadas geográficas en grados) a la ubicación del restaurante de la orden:

$$d(c, o) = \sqrt{(\phi_c - \phi_r)^2 + (\psi_c - \psi_r)^2} \quad (3.7)$$

donde  $(\phi_c, \psi_c)$  es la ubicación actual del courier y  $(\phi_r, \psi_r)$  la del restaurante. Se selecciona  $c^* = \arg \min_{c \in \mathcal{C}_{disp}} d(c, o)$ . Esta distancia opera sobre grados decimales y se utiliza únicamente como criterio de ordenamiento relativo; la conversión a metros no es necesaria porque el ranking se preserva.

3. **Estimación de ruta por Haversine:** La duración de la ruta  $c^*.location \rightarrow r_o \rightarrow \ell_o$  (courier  $\rightarrow$  restaurante  $\rightarrow$  cliente) se estima mediante la distancia de **Haversine** entre cada par de puntos consecutivos, convertida a tiempo con una velocidad constante de 320 m/min. En contraste con la política RH, que consulta OSRM para obtener tiempos sobre la red vial real, FCFS no dispone de información de calles, sentidos ni velocidades diferenciales. La duración resultante determina el *completion time*.
4. **Compromiso inmediato:** Se asigna la ruta al courier como compromiso **final** (sin etapa parcial), se calcula el tiempo de finalización como  $t_{completion} = t + \text{duration}$ , y la orden pasa al estado **assigned**.

Las órdenes se procesan secuencialmente en el orden en que aparecen en la lista de órdenes listas. Esto implica que las primeras órdenes procesadas obtienen los couriers más cercanos, mientras que las últimas pueden recibir couriers más lejanos, introduciendo un sesgo de orden de iteración.

### 3. 4.2. Características y limitaciones

La política FCFS presenta las siguientes propiedades que la distinguen de la política Rolling Horizon:

- **Sin agrupamiento:** Cada orden genera una ruta individual (`bundle_size = 1`). El courier viaja al restaurante, recoge una sola orden y la entrega antes de quedar disponible para la siguiente asignación.
- **Sin optimización de matching:** La asignación se basa únicamente en proximidad geográfica instantánea, sin considerar el horizonte de asignación  $\Delta_u$ , la frescura de las órdenes, ni la carga futura del sistema.
- **Compromiso directo:** No existe la etapa de compromiso parcial (*tentative assignment*). El courier se compromete de inmediato con la ruta completa pickup  $\rightarrow$  delivery.
- **Enrutamiento simplificado (Haversine):** Todas las estimaciones de distancia y tiempo de viaje se calculan mediante la fórmula de Haversine a 320 m/min, sin consultar la red vial. Esto sobreestima o subestima los tiempos reales según la topología local (ej. calles de un solo sentido, bahía de La Paz), pero replica fielmente la información de que dispone una plataforma sin motor de enrutamiento.
- **Activación:** La política se activa estableciendo las variables de entorno `FCFS_POLICY=1` y `USE_EUCLIDEAN=1` antes de ejecutar la simulación. Sin estas variables, el motor utiliza Rolling Horizon con OSRM por defecto.

Estas simplificaciones hacen de FCFS una cota inferior práctica: cualquier mejora que la política Rolling Horizon logre sobre FCFS puede atribuirse a la combinación de enrutamiento vial realista (OSRM) y los mecanismos de agrupamiento y optimización global que introduce.

### 3. 5. Política Rolling Horizon

La política de **Rolling Horizon** (RH) constituye la heurística central de este trabajo, basada en el marco propuesto por Reyes et al. [13]. A diferencia de FCFS, RH agrupa órdenes del mismo restaurante en *agrupamientos* (bundles), resuelve un problema de asignación bipartita courier-agrupamiento mediante el Algoritmo Húngaro, y emplea un esquema de compromiso en dos etapas que permite reoptimizar asignaciones entre épocas. Esta sección describe los cuatro componentes secuenciales que se ejecutan en cada época de asignación.

#### 3. 5.1. Cálculo del tamaño objetivo de agrupamiento

Al inicio de cada época, el sistema estima el tamaño objetivo de agrupamiento  $Z_t$  como el ratio entre las órdenes próximas a estar listas y los couriers que estarán disponibles en el horizonte inmediato [13]:

$$Z_t = \left\lceil \frac{|\{o \in \mathcal{O}(t) : t_{ready}(o) \leq t + \Delta_1\}|}{|\{c \in \mathcal{C}_{disp}(t) : e_c^{avail} \leq t + \Delta_2\}|} \right\rceil \quad (3.8)$$

donde  $\mathcal{O}(t)$  es el conjunto de órdenes reveladas al tiempo  $t$  (estado **ready**),  $e_c^{avail}$  es el instante en que el courier  $c$  queda libre (fin de ruta actual, o  $t$  si está inactivo),  $\Delta_1 = 20$  min es el horizonte de anticipación para órdenes listas y  $\Delta_2 = 20$  min es el horizonte para disponibilidad de couriers. El valor se acota superiormente por `MAX_TARGET_BUNDLE_SIZE` = 5 para evitar agrupamientos irrealistamente grandes bajo picos de demanda. Si no hay couriers disponibles en el horizonte, se utiliza cualquier courier activo en turno como denominador de respaldo; si aun así no hay couriers,  $Z_t = 1$ .

Este indicador adapta dinámicamente la consolidación de órdenes al nivel de saturación instantáneo: cuando la demanda supera la oferta de couriers,  $Z_t$  crece y se forman agrupamientos más grandes; cuando hay holgura,  $Z_t \rightarrow 1$  y el comportamiento converge hacia asignaciones individuales similares a FCFS.

#### 3. 5.2. Generación de agrupamientos por restaurante

Para cada restaurante con órdenes pendientes, el módulo `bundling.py` construye agrupamientos mediante un procedimiento de **inserción paralela con mejora local** (*parallel insertion with remove-reinsert improvement*):

1. **Filtrado:** Se seleccionan las órdenes del restaurante con estado **ready** cuyo *ready time* cae dentro del horizonte de asignación ( $t_{ready} \leq t + \Delta_u$ ).
2. **Inicialización:** Se ordenan las órdenes por  $t_{ready}$  ascendente y se crean  $m_r$  agrupamientos vacíos, donde:

$$m_r = \min\left(\left\lceil \frac{n_{orders}}{Z_t} \right\rceil, |\mathcal{C}_{disp}|\right) \quad (3.9)$$

garantizando que no se generen más agrupamientos que couriers disponibles.

3. **Inserción:** Para cada orden, se evalúan todas las posiciones de inserción en todos los agrupamientos existentes. El costo de cada inserción candidata se calcula mediante:

$$\text{costo}(B, o, p) = T_{ruta}(B \oplus_p o) + \beta \cdot T_{servicio}(B \oplus_p o) \quad (3.10)$$

donde  $B \oplus_p o$  denota la inserción de la orden  $o$  en la posición  $p$  del agrupamiento  $B$ ,  $T_{ruta}$  es la duración OSRM de la ruta resultante,  $T_{servicio}$  es el tiempo de servicio acumulado, y  $\beta = 0.1$  es la penalización de frescura. Se selecciona la combinación  $(B^*, p^*)$  que minimiza el costo. Si un agrupamiento ya alcanza  $Z_t$  órdenes, solo se permite la inserción si mejora la eficiencia (tiempo promedio por orden). El tamaño máximo absoluto es  $\text{MAX\_BUNDLE\_SIZE} = 4$ .

4. **Mejora local (*remove-reinsert*):** Se ejecutan 2 iteraciones de un procedimiento de mejora: para cada orden en cada agrupamiento, se extrae temporalmente y se evalúa su reinserción en todas las posiciones de todos los agrupamientos. Si existe una posición con menor costo que la actual, se realiza el movimiento. Este paso permite corregir asignaciones subóptimas del procedimiento greedy inicial.

Los agrupamientos vacíos resultantes se descartan antes de pasar a la fase de asignación.

### 3. 5.3. Clasificación por prioridad y asignación bipartita

Una vez generados todos los agrupamientos de todos los restaurantes, se clasifican en tres grupos de prioridad según Reyes et al. [13]:

- **Grupo I** (urgente): El tiempo de entrega más temprano posible excede  $a_o + \tau$  para la orden más antigua del agrupamiento, donde  $a_o$  es el *placement time* y  $\tau = 40$  min es el objetivo de Click-to-Door. Ningún courier puede cumplir la meta de servicio.

- **Grupo II** (retrasado): El agrupamiento no pertenece al Grupo I, pero ningún courier puede llegar al restaurante antes del *ready time* del agrupamiento. Existe retraso en la recolección pero la meta de entrega aún es alcanzable.
- **Grupo III** (normal): El agrupamiento puede ser recogido y entregado dentro de los objetivos de servicio.

La asignación se resuelve en **tres rondas secuenciales** ( $I \rightarrow II \rightarrow III$ ), priorizando los agrupamientos más urgentes. En cada ronda, se construye una matriz de costos  $\mathbf{W} \in \mathbb{R}^{|\mathcal{C}_{disp}| \times |B_g|}$  donde cada entrada es el peso de matching:

$$w_{cj} = \frac{|B_j|}{T_{delivery}(c, B_j)} - \theta \cdot (t_{pickup}(c, B_j) - t_{ready}(B_j)) - P_g \quad (3.11)$$

donde  $|B_j|$  es el número de órdenes en el agrupamiento  $j$ ,  $T_{delivery}(c, B_j)$  es el tiempo total de entrega expresado en horas,  $t_{pickup}(c, B_j) - t_{ready}(B_j)$  es el retraso de recolección en minutos,  $\theta = 0.1$  es un coeficiente de penalización por retraso en la recolección, y  $P_g$  es una penalización de prioridad que actúa como refinamiento *intra-grupo*: dado que la asignación ya se resuelve en tres rondas secuenciales por grupo de urgencia,  $P_g$  opera como criterio de desempate dentro de cada ronda —  $P_g = 100$  si el par  $(c, B_j)$  excede  $\tau_{max}$ ,  $P_g = 50$  si presenta retraso en la recolección, y  $P_g = 0$  en caso contrario. Si un par courier-agrupamiento es infactible (sin ruta OSRM), se asigna un costo prohibitivo ( $10^9$ ).

La matriz se convierte a costos ( $c_{ij} = -w_{cj}$ ) y se resuelve mediante `scipy.optimize.linear_sum_assignment` que implementa el **Algoritmo Húngaro** con complejidad  $O(n^3)$ . Los couriers asignados a celdas con costo prohibitivo se descartan.

### 3. 5.4. Compromiso en dos etapas

Para cada par  $(c, B)$  seleccionado por el Algoritmo Húngaro, se aplica el **compromiso aditivo en dos etapas** (*two-stage additive commitment*) descrito en Reyes et al. [13]:

1. **Excepción de espera prolongada:** Si alguna orden del agrupamiento lleva lista más de  $x = 10$  min ( $t - t_{ready}(o) > x$ ), se fuerza un compromiso **final** independientemente de las demás condiciones. Esto evita que órdenes queden indefinidamente sin asignar.



2. **Compromiso final:** Si el tiempo estimado de llegada del courier al restaurante cae dentro de la época actual ( $\hat{t}_{arrival} \leq t + f$ ) y todas las órdenes del agrupamiento estarán listas dentro de la época ( $t_{ready}(o) \leq t + f, \forall o \in B$ ), se asigna la ruta completa:  $\text{courier} \rightarrow \text{restaurante} \rightarrow \text{entrega}_1 \rightarrow \dots \rightarrow \text{entrega}_n$ . En la implementación,  $\hat{t}_{arrival}$  se estima como la mitad de la duración OSRM de la ruta completa, lo que introduce un sesgo optimista que favorece compromisos finales sobre parciales. El courier queda comprometido hasta  $t_{completion}$ .
3. **Compromiso parcial:** Si no se cumplen las condiciones anteriores, se asigna únicamente la ruta **inbound** ( $\text{courier} \rightarrow \text{restaurante}$ ). El courier comienza a desplazarse hacia el restaurante, pero en la siguiente época el sistema puede reoptimizar la asignación: actualizar el agrupamiento, cambiar de courier, o añadir/quitar órdenes.
4. **Ignorar:** Si no es posible calcular la ruta completa del agrupamiento (fallo de OSRM o conexión), o si no se cumplen las condiciones de compromiso final y además la ruta inbound tampoco puede calcularse, la asignación se descarta y el agrupamiento queda disponible para la siguiente época.

Este mecanismo *lazy* de compromiso evita fijar prematuramente rutas con información parcial. Un courier con compromiso parcial aparece como ocupado ( $\text{current\_route} \neq \emptyset$ ), pero al completar su tramo inbound en la fase de actualización, queda libre de nuevo. En la práctica, esto proporciona una ventana de tiempo adicional para que más órdenes se revelen y puedan consolidarse en un agrupamiento más eficiente.

### 3. 5.5. Resumen del flujo RH por época

La Figura 3.2 resume el flujo completo de la política Rolling Horizon en cada época de asignación.

## 3. 6. Diseño experimental

La evaluación comparativa de las políticas FCFS y Rolling Horizon se estructura como un experimento pareado sobre instancias sintéticas generadas según el proceso descrito en la Sección 3.2. El script `run_experiments.py` orquesta la ejecución completa.

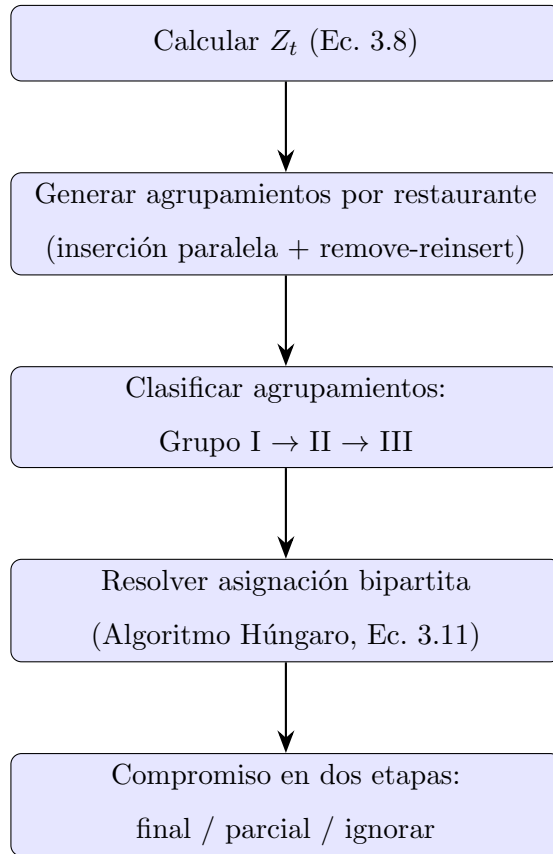


Figura 3.2: Flujo de la política Rolling Horizon en cada época de asignación.

### 3. 6.1. Variable de tratamiento: saturación de flota

La variable independiente es la *razón de saturación*  $r$ , definida como el número objetivo de pedidos por repartidor. A partir de  $r$  se dimensiona la flota según la Ecuación 3.3, manteniendo fijo el volumen de pedidos  $|\mathcal{O}|$ . Un valor alto de  $r$  produce una flota reducida con alta carga individual, mientras que un valor bajo genera capacidad holgada.

### 3. 6.2. Escenarios

Se definen tres escenarios que cubren desde operación holgada hasta saturación media (Tabla 3.3). En todos ellos  $n_{\min} = 10$ .

Tabla 3.3: Escenarios experimentales de saturación ( $|\mathcal{O}| \approx 1\,038$  pedidos).

| Escenario | Nivel    | $r$ | $ \mathcal{C} $ | Descripción                                |
|-----------|----------|-----|-----------------|--------------------------------------------|
| S1        | Baja     | 18  | 58              | Línea base; $r$ igual al valor por defecto |
| S2        | Moderada | 26  | 40              | Reducción moderada de flota                |
| S3        | Media    | 35  | 30              | Punto medio de saturación                  |

### 3. 6.3. Comparación pareada

Para cada combinación de escenario y semilla, ambas políticas se ejecutan sobre la *misma* instancia de pedidos y la *misma* flota de repartidores. La simulación se invoca en proceso (*in-process*): la función `_run_policy` ajusta el parámetro  $r$  del escenario y activa la política correspondiente mediante la variable de entorno `FCFS_POLICY` (presente  $\rightarrow$  FCFS; ausente  $\rightarrow$  Rolling Horizon). Este diseño pareado elimina la variabilidad inter-instancia y aísla el efecto de la política de asignación.

### 3. 6.4. Replicación y análisis de sensibilidad

El marco experimental admite dos ejes adicionales de variación:

- **Semillas múltiples.** Se emplean diez semillas aleatorias ( $s \in \{2025, 2026, \dots, 2034\}$ ), cada una de las cuales genera una instancia distinta con la misma distribución subyacente.

Esto permite estimar la variabilidad estadística de los indicadores y reportar promedios e intervalos de confianza sobre diez réplicas independientes.

- **Tamaño máximo de agrupamiento.** Se puede variar el parámetro `MAX.BUNDLE.SIZE` entre ejecuciones para evaluar su efecto sobre la eficiencia de la política RH, manteniendo el resto de la configuración constante.

El número total de ejecuciones de simulación es  $2 \times |S| \times |Seeds| \times |B|$ , donde  $|S|$  es el número de escenarios,  $|Seeds|$  las semillas y  $|B|$  los valores de tamaño de agrupamiento evaluados; el factor 2 corresponde a las dos políticas. En la configuración base ( $|S| = 3$ ,  $|Seeds| = 10$ ,  $|B| = 1$ ) se realizan  $2 \times 3 \times 10 \times 1 = 60$  ejecuciones.

### 3. 6.5. Almacenamiento de resultados

Cada corrida del marco genera un directorio con marca temporal bajo `results/experiments/`. Para cada escenario se almacenan cuatro archivos CSV (resultados de pedidos y repartidores, por política). Al finalizar todos los escenarios, el script produce:

- `experiment_comparison.csv`: tabla completa con los KPIs de ambas políticas por escenario y la mejora porcentual  $\Delta \% = 100 \cdot (RH - FCFS) / |FCFS|$ .
- `experiment_summary.csv`: subconjunto de indicadores clave (CTD promedio, costo por pedido, pedidos por repartidor por hora, utilización, cumplimiento SLA y distancia por pedido).
- `experiment_metadata.json`: parámetros de la ejecución (semillas, escenarios, tamaños de agrupamiento, marca temporal).

## 3. 7. Métricas de evaluación

El desempeño de cada política se cuantifica mediante 21 indicadores clave (*KPIs*) agrupados en cinco dimensiones: calidad de servicio, cobertura, eficiencia operativa, eficiencia de agrupamiento y costo. Las definiciones se basan en las métricas propuestas por Reyes et al. [13] y se implementan en la función `calculate_kpis` de `run_experiments.py`.

### 3. 7.1. Calidad de servicio

El indicador principal es el **Click-to-Door** (CtD), definido como el tiempo transcurrido desde la colocación de la orden hasta su entrega al cliente:

$$\text{CtD}_o = t_{\text{delivery},o} - a_o$$

Se reportan la media y los percentiles 50, 90 y 95. Adicionalmente se calculan:

- **Ready-to-Pickup** (RtP): tiempo desde que la orden está lista en el restaurante hasta que el repartidor la recoge ( $\text{RtP}_o = t_{\text{pickup},o} - t_{\text{ready},o}$ ). Se reportan la media y el percentil 90.
- **Ready-to-Door** (RtD): tiempo desde la disponibilidad del pedido hasta su entrega ( $\text{RtD}_o = t_{\text{delivery},o} - t_{\text{ready},o}$ ). Se reportan la media y el percentil 90.
- **Cumplimiento SLA**: porcentaje de órdenes entregadas con  $\text{CtD}_o \leq \tau = 40$  min.
- **CTD Overage**: exceso promedio respecto al objetivo,  $\frac{1}{n} \sum \max(0, \text{CtD}_o - \tau)$ .

### 3. 7.2. Cobertura

- **% Pedidos no entregados**: fracción de órdenes que no alcanzan el estado `delivered` al cierre de la simulación, calculada sobre el total de órdenes de la instancia.

### 3. 7.3. Eficiencia operativa

Estas métricas se derivan del CSV de repartidores, que registra distancia recorrida (km), tiempo de conducción (min) y duración del turno (h) por repartidor.

- **Pedidos por repartidor por hora**:  $\sum n_{\text{del},c} / \sum h_{\text{shift},c}$ .
- **Agrupamientos por hora**:  $\sum b_c / \sum h_{\text{shift},c}$ , donde  $b_c$  son los agrupamientos recogidos por el repartidor  $c$ .
- **Utilización (%)**: fracción del turno que el repartidor estuvo conduciendo,  $100 \cdot \sum t_{\text{drive},c} / \sum h_{\text{shift},c}$ .
- **Distancia total** (km): suma de las distancias de ruta de todos los repartidores.

- **Distancia por pedido** (km): distancia total dividida entre el número de pedidos entregados.

### 3. 7.4. Eficiencia de agrupamiento

- **Tamaño promedio de agrupamiento**: media del campo `bundle_size` de las órdenes entregadas.
- **% Pedidos en multi-agrupamiento**: proporción de órdenes entregadas cuyo `bundle_size`  $> 1$ ; refleja en qué medida la política logra consolidar entregas.

### 3. 7.5. Costo

El modelo de compensación sigue el esquema dual descrito en la Ecuación 3.5: cada repartidor recibe el máximo entre la ganancia por entregas ( $p_1 \cdot n_{orders}$ ) y el pago mínimo garantizado ( $p_2 \cdot h_{shift}$ ). A partir de este modelo se obtienen:

- **Compensación total** (\$): suma de las compensaciones individuales de todos los repartidores.
- **Costo por pedido** (\$): compensación total dividida entre el número de pedidos entregados.
- **Fracción con compensación mínima**: proporción de repartidores cuya compensación iguala el pago mínimo garantizado, indicando subutilización de la flota.

### 3. 7.6. Resumen de indicadores

La Tabla 3.4 presenta los 21 indicadores con su definición abreviada y la dimensión a la que pertenecen.

Tabla 3.4: Indicadores clave de desempeño (KPIs) evaluados.

| #  | Dimensión | Indicador      | Definición                               |
|----|-----------|----------------|------------------------------------------|
| 1  | Calidad   | Avg CtD        | Media del Click-to-Door (min)            |
| 2  | Calidad   | P50 CtD        | Mediana del CtD                          |
| 3  | Calidad   | P90 CtD        | Percentil 90 del CtD                     |
| 4  | Calidad   | P95 CtD        | Percentil 95 del CtD                     |
| 5  | Calidad   | Avg RtP        | Media del Ready-to-Pickup (min)          |
| 6  | Calidad   | P90 RtP        | Percentil 90 del RtP                     |
| 7  | Calidad   | Avg RtD        | Media del Ready-to-Door (min)            |
| 8  | Calidad   | P90 RtD        | Percentil 90 del RtD                     |
| 9  | Calidad   | SLA Compliance | % de órdenes con $CtD \leq 40$ min       |
| 10 | Calidad   | CTD Overage    | Exceso promedio sobre $\tau$ (min)       |
| 11 | Cobertura | % Undelivered  | Órdenes no entregadas / total            |
| 12 | Operativa | Orders/C/Hr    | Pedidos por repartidor por hora          |
| 13 | Operativa | Bundles/Hr     | Agrupamientos por hora                   |
| 14 | Operativa | Utilización    | % del turno conduciendo                  |
| 15 | Operativa | Dist. total    | Kilómetros recorridos (flota)            |
| 16 | Operativa | Dist./Pedido   | km por pedido entregado                  |
| 17 | Agrup.    | Avg Bundle     | Tamaño medio de agrupamiento             |
| 18 | Agrup.    | % Multi        | % órdenes en agrup. $> 1$                |
| 19 | Costo     | Comp. total    | Compensación total (\$)                  |
| 20 | Costo     | Costo/Pedido   | Compensación / pedidos entregados (\$)   |
| 21 | Costo     | Frac. Mín.     | Fracción de repartidores con pago mínimo |

# Capítulo 4

## Resultados

4. 1. Configuración del experimento

4. 2. Calidad de servicio

4. 3. Validación estadística

4. 3.1. Interpretación de resultados estadísticos

4. 3.2. Verificación de la hipótesis

4. 3.3. Limitaciones de los resultados



## Capítulo 5

## Conclusiones

# Bibliografía

- [1] Jan Karel Lenstra and Alexander H. G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2):221–227, 1981. doi: 10.1002/net.3230110204.
- [2] G. B. Dantzig and J. H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959.
- [3] Jose Cáceres-Cruz, Pol Arias, Daniel Guimarans, Daniel Riera, and Angel A. Juan. Rich vehicle routing problem: Survey. *ACM Computing Surveys*, 47(2):32:1–32:28, December 2014. doi: 10.1145/2666003.
- [4] Gilbert Laporte. Fifty years of vehicle routing. *Transportation Science*, 43(4):408–416, 2009. doi: 10.1287/trsc.1090.0301.
- [5] Reza Jazemi, Ensieh Alidadiani, Kwangseog Ahn, and Jaejin Jang. A review of literature on vehicle routing problems of last-mile delivery in urban areas. *Applied Sciences*, 13(2413015), 2023.
- [6] Harilaos N. Psaraftis. A dynamic programming approach for the single-vehicle many-to-many immediate-request dial-a-ride problem. *Transportation Science*, 14(2):130–154, 1980. doi: 10.1287/trsc.14.2.130.
- [7] Jean-François Cordeau and Gilbert Laporte. The dial-a-ride problem: Models and algorithms. *Annals of Operations Research*, 153:29–46, 2007. doi: 10.1007/s10479-007-0170-8.
- [8] Soumia Ichoua, Michel Gendreau, and Jean-Yves Potvin. Exploiting knowledge about future demands for real-time vehicle dispatching. *Transportation Science*, 40(2):211–225, 2006. doi: 10.1287/trsc.1050.0114.

- [9] Jesper Larsen. The dynamic vehicle routing problem: Survey of classifications. *Computers & Operations Research*, 27(10):103–122, 2000. doi: 10.1016/S0305-0548(99)00100-X.
- [10] Victor Pillac, Michel Gendreau, Christelle Gu  ret, and Andr  s L. Medaglia. A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1):1–11, 2013. doi: 10.1016/j.ejor.2012.08.015.
- [11] Rafael H. M. Pereira, Marcus Saraiva, Daniel Herszenhut, Carlos Kaue Braga, and Matthew Wigginton Conway. r5r: Rapid realistic routing on multimodal transport networks with R5 in R. *Findings*, 2021. doi: 10.32866/001c.21262. URL <https://findingspress.org/article/21262-r5r-rapid-realistic-routing-on-multimodal-transport-networks-with-r5-in-r>.
- [12] Burak Boyacı, Thu Huong Dang, and Adam N. Letchford. Vehicle routing on road networks: How good is euclidean approximation? *Computers & Operations Research*, 129:105197, 2021. doi: 10.1016/j.cor.2020.105197.
- [13] Damian Reyes, Alan Erera, Martin Savelsbergh, Sagar Sahasrabudhe, and Ryan O’Neil. The meal delivery routing problem. *Optimization Online*, March 2018. URL [http://www.optimization-online.org/DB\\_FILE/2018/03/6512.pdf](http://www.optimization-online.org/DB_FILE/2018/03/6512.pdf).
- [14] Miguel Andres Figliozzi. Analysis of the efficiency of urban commercial vehicle tours: Data collection, methodology, and policy implications. *Transportation Research Part B: Methodological*, 41(9):1014–1032, 2007. doi: 10.1016/j.trb.2007.04.006.
- [15] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2, 2014. URL <http://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
- [16] Lin Wu, Haotian Wen, Hai Hu, Yong Wang, Yiran Li, Jian Wang, Jieping Ye, Haoyu Zhang, and Meng Xu. Lade: The first comprehensive last-mile delivery dataset from industry. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM ’23)*, page 2619–2628, 2023. doi: 10.1145/3583780.3614820.
- [17] Mordechai Haklay. How good is volunteered geographical information? a comparative study

- of openstreetmap and ordnance survey datasets. *Environment and Planning B: Planning and Design*, 37(4):682–703, 2010. doi: 10.1068/b35097.
- [18] Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Proceedings of the 7th International Workshop on Experimental Algorithms (WEA 2008)*, volume 5038 of *Lecture Notes in Computer Science*, pages 319–333, Berlin, Heidelberg, 2008. Springer. doi: 10.1007/978-3-540-68552-4\_24.